



PLURALSIGHT

CG - AI Foundations

Welcome!



Varoon Sahgal
Instructor, Pluralsight



Setting Expectations

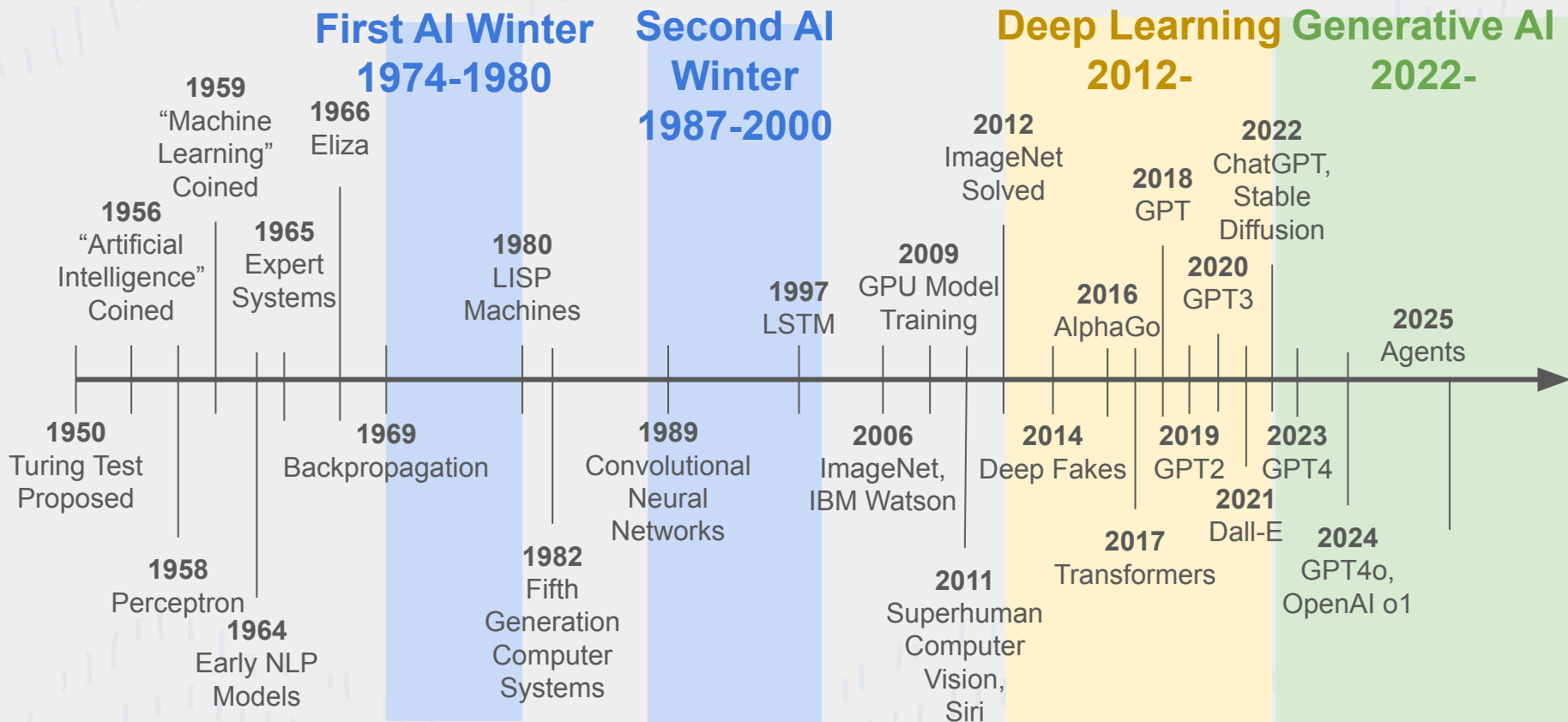
- A lot of the hands-on work we do will be in Google Colab (we will go over this shortly)
- We will have exercises for you in Colab

You experience our impact on a daily basis!

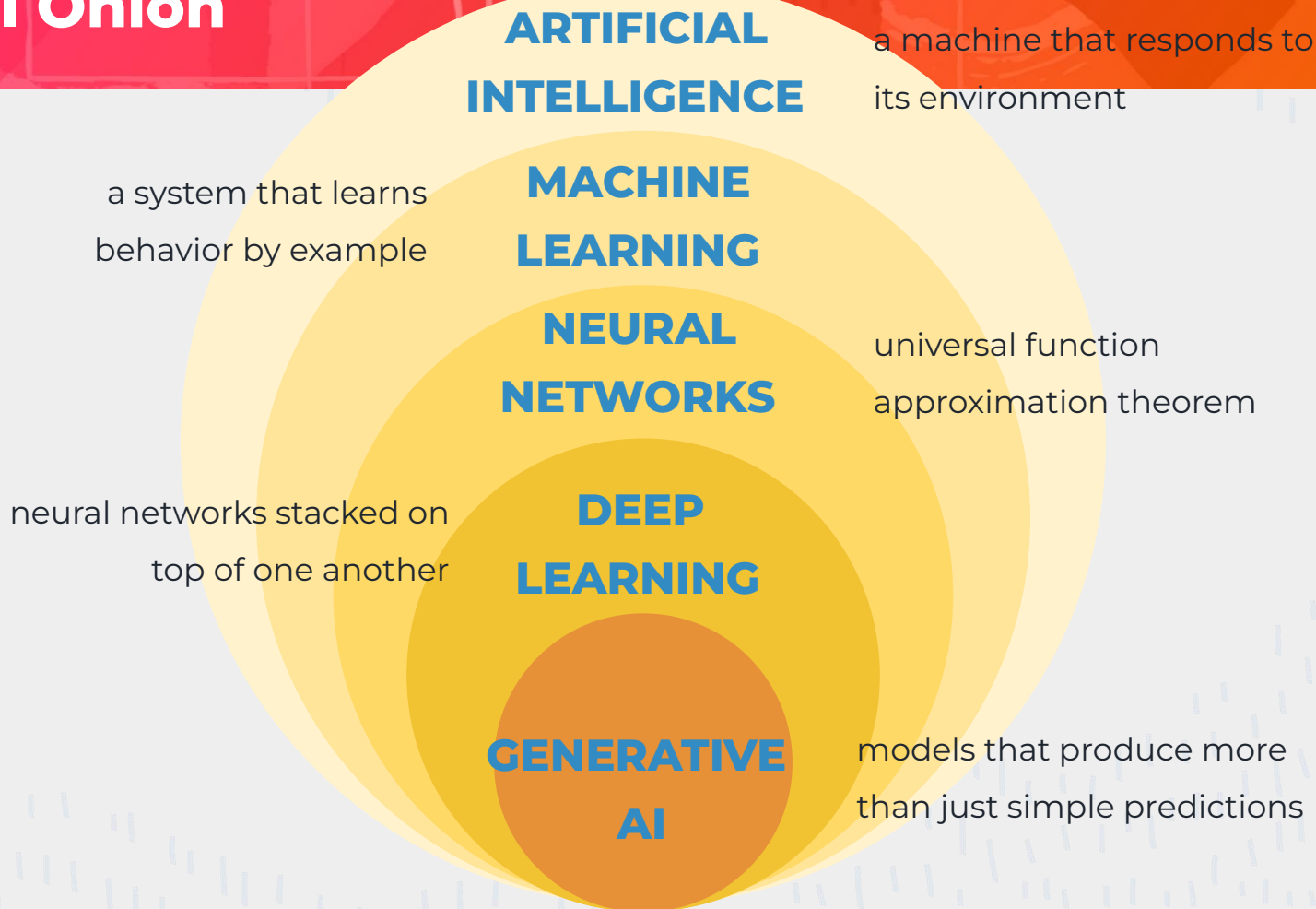


FOUNDATIONS OF AI AND MACHINE LEARNING

A Brief History of AI



The AI Onion



Machine Learning vs. Rule-Based Systems

Rule-Based:

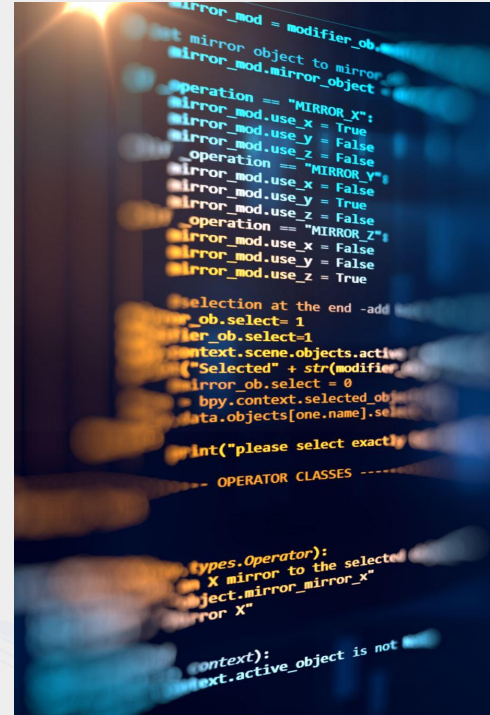
- Requires explicit programming, difficult to scale.

ML-Based:

- Learns from data patterns, adapts over time.

Example:

- Spam filters – from keyword rules to adaptive ML models.



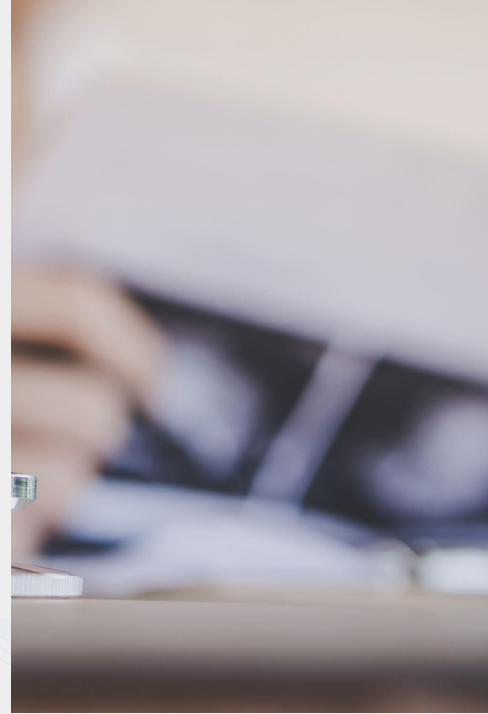
Rule-Based Decision Making

Expert knowledge representation

- Human decision making often follows rules
- Example: medical diagnosis based on symptoms

Challenges:

- Certain tasks, such as object recognition in images or videos, can't be entirely represented by rules.



Rule-based systems: examples and limitations



Location: Is the purchase outside of the common area where the user usually makes their purchases? How far? Is the purchase unusual for that location?



Frequency: Is this account being used more frequently than ever before? Are those purchases unusual for the user?



Unknown Sender/Receiver: Is the user receiving unusually large amounts from unfamiliar accounts? Are those accounts newly created? Is someone creating several accounts from the same IP address in a short period of time and sending money?



- Credit card fraud detection

- <https://fraud.net/d/rules-based-fraud-detection/>

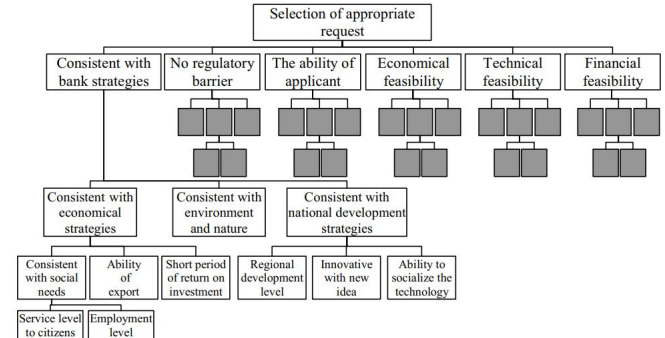
- Loan application approval

- https://www.researchgate.net/publication/220841474_Presenting_a_Rule_Based_Loan_Evaluation_Expert_System



Problem:

- There are too many rules and "exceptions," making it difficult to account for all possible scenarios.
- The rules often create a "continuum."



Self-driving vehicles

- What might a rule-based solution look like?
- Would it be effective?
- If not, why?



Image classification

- What might an rule-based solution look like?
- Would it be effective?
- If not, why?



<https://www.kaggle.com/c/dogs-vs-cats>

Language translation

- What might an instruction-based solution look like?
- Would it be effective?
- If not, why?



Machine Learning

- “The field of study that gives computers the ability to learn without being explicitly programmed” (Arthur Samuel, 1959)



All Machine Learning involves a model

*All machine learning involves a model at its core. The model is what **learns patterns from data** and then makes predictions, classifications, or decisions.*

All Machine Learning involves a model

A model is a **mathematical function or structure** that:

- Takes input (X)
- Learns from data
- Outputs a prediction ($\hat{Y}$)

Common ML model types

Common Machine Learning Model Types & Applications

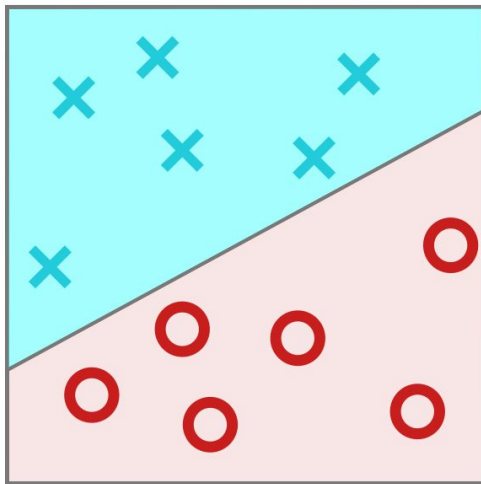
Model Type	Examples	Common Applications
Linear Models	Linear/Logistic Regression	Price prediction, churn detection
Tree-Based Models	Decision Trees, Random Forest	Credit scoring, fraud detection
Ensemble Methods	Gradient Boosting (e.g., XGBoost)	Customer segmentation, ranking models
Neural Networks	MLPs, CNNs, RNNs, Transformers	Image recognition, NLP (e.g., ChatGPT)
Clustering Models	K-Means, DBSCAN	Market segmentation, anomaly detection
Dimensionality Reduction	PCA, t-SNE	Data visualization, noise filtering

Is a model just an application?

- No, the application is a wrapper around the model
- Think of the model as an engine, but the application is the car
 - the model by itself has no real utility, but when put in a larger application it has meaning
 - a. - Much like an engine without a car has no utility..
 - b.

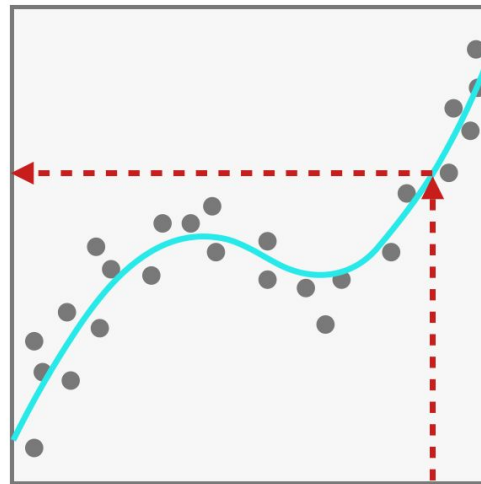
Classification vs. Regression

Classification Groups observations into "classes"



Here, the line classifies the observations into X's and O's

Regression predicts a numeric value



Here, the fitted line provides a predicted output, if we give it an input

[Image credit](#)

Perception vs. Generation

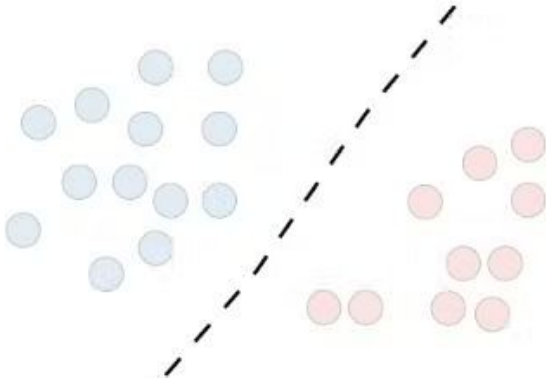
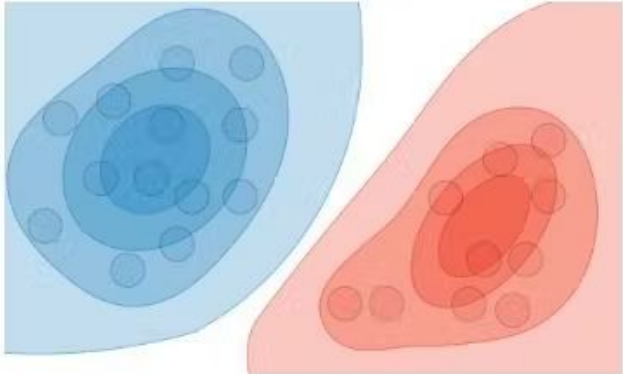
PERCEPTION

Until recently, most AI you've probably heard about were for **perception**. These include things like FaceID, speech recognition, self driving cars, and the like. They try to mimic vision, hearing, or other senses to detect something. These typically take **complex inputs and produce simpler outputs**.

GENERATION

Generative AI is about producing things that mirror real world data in some useful way. This can be chatbots which can hold realistic conversations, image generation that produce realistic pictures, to name a few. These typically take **simpler inputs and produce complex outputs**.

Discriminative vs. Generative

	Discriminative model	Generative model
Goal	Directly estimate $P(y x)$	Estimate $P(x y)$ to then deduce $P(y x)$
What's learned	Decision boundary	Probability distributions of the data
Illustration	 An illustration of a discriminative model. It shows two clusters of data points: blue circles on the left and red circles on the right. A dashed black line, representing the decision boundary, separates the two clusters.	 An illustration of a generative model. It shows two overlapping probability density functions (PDFs). The left PDF is blue and contains several blue data points. The right PDF is red and contains several red data points. The PDFs represent the estimated probability distributions for each class.
Examples	Regressions, SVMs	GDA, Naive Bayes

[Image credit](#)

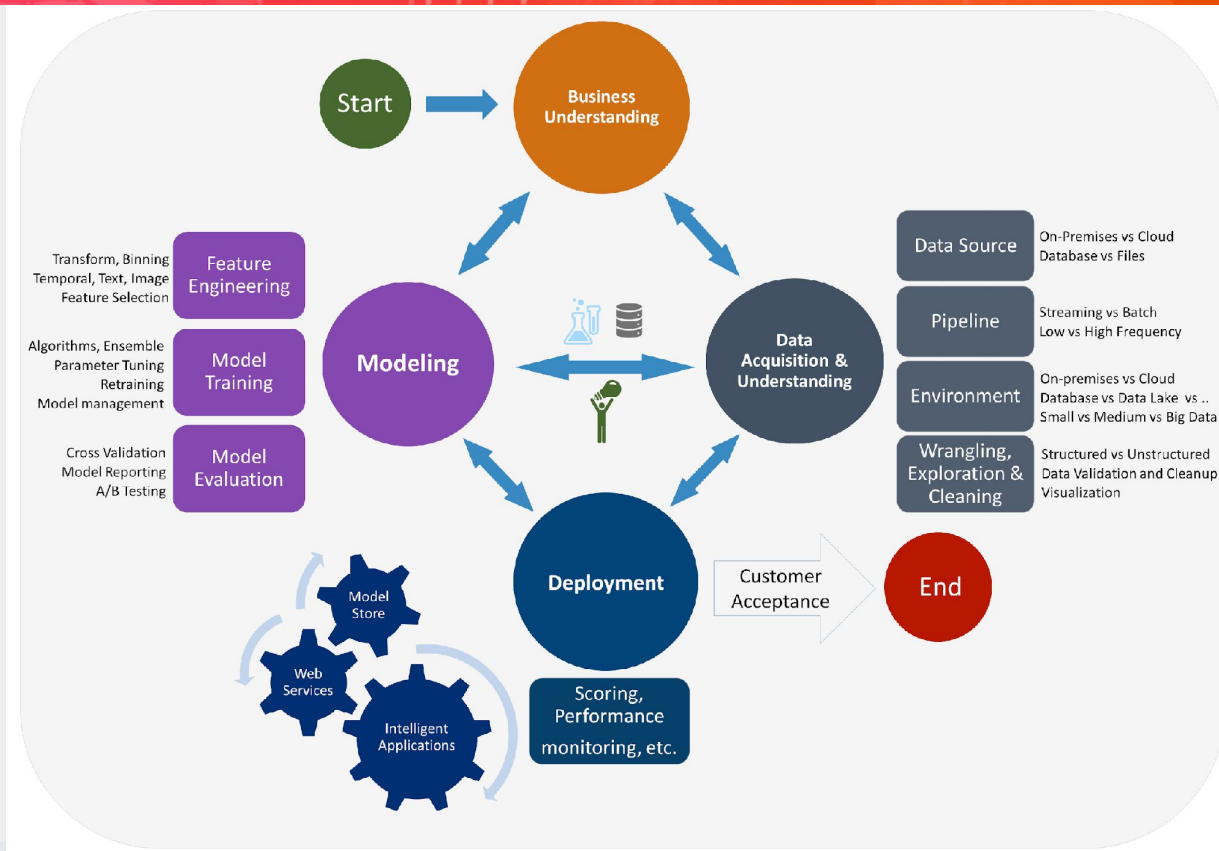
A note on randomness

At their core, all generative models themselves are **deterministic**.

However, fully deterministic output is often not desirable.

Randomness is applied **external** to the model in various ways.

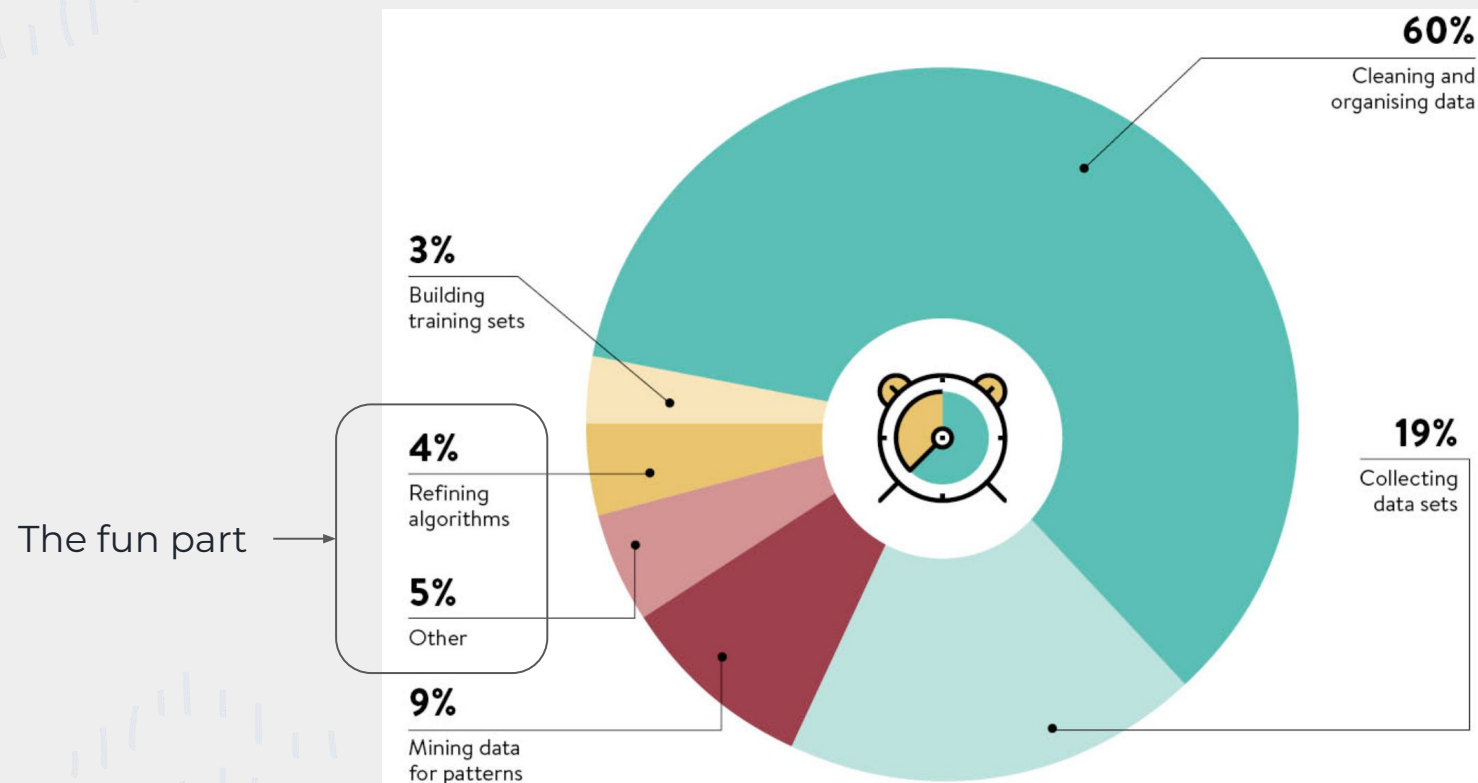
Data Science Project Life Cycle



Data Science Project Time Allocation



Data Science Project Time Allocation



Model Overfitting



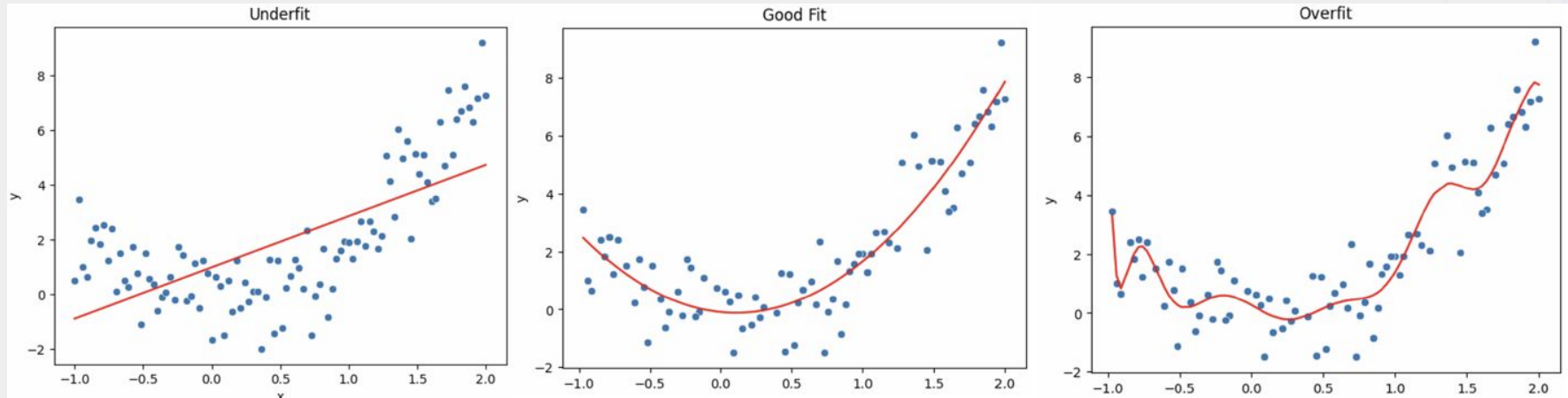
Overfitting: the model shows excellent performance in development but fails to predict on new data effectively.



An overfit model may learn the 'noise' in the training data rather than the true underlying patterns.

Underfit and Overfit

- Example: which model (red) has the best ability to generalize the training data (blue)?



Model Performance Metrics

quantitative measures used to evaluate how well a machine learning model makes predictions



help assess the accuracy, reliability, and efficiency of the model in achieving its objective

Regression model performance metrics

$$\text{MAE} = \frac{1}{n} \sum |y_i - \hat{y}_i|$$

Mean Absolute Error (MAE):

Average absolute difference between predictions and actual values.

$$\text{MSE} = \frac{1}{n} \sum (y_i - \hat{y}_i)^2$$

Mean Squared Error (MSE):

Average squared difference between predictions and actual values.

$$R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$$

R-Squared:

Proportion of variance in the target variable explained by the model.

Varies between 0 (bad) and 1 (ideal)

Understanding Mean Squared Error (MSE)

MSE doesn't have an absolute baseline.

- It's impossible to determine whether an MSE is 'good' or 'bad' based on an isolated model experiment.

MSE is used for model selection.

- A lower MSE indicates better predictive accuracy when comparing multiple models.

MSE represents the variance of the data that is not explained by the model.

- For a well-fitted model, the MSE is significantly lower than the total variance of the target variable.

Classification model performance metrics

- Confusion Matrix
- Accuracy
- Precision
- Recall
- F1 score
- ROC Curve and AUC

		Predicted labels	
		Positive	Negative
True labels	Positive (P)	<u>True positive</u> (TP)	<u>False negative</u> (FN)
	Negative (N)	<u>False positive</u> (FP)	<u>True negative</u> (TN)

Read more:

<https://encord.com/blog/classification-metrics-accuracy-precision-recall/>

Accuracy: The Basic Metric

- Accuracy measures the proportion of correct.
- Limitations: Accuracy can be misleading if there's an imbalance in the data

$$\text{Accuracy} = \frac{TP + TN}{TP + FP + TN + FN}$$

- Example:
 - The training set has 1M transactions, only 1,000 of them are labeled “fraud”.
 - The “dummy” model always predicts “no fraud” (999,000 out of 1M are correct)
 - The accuracy is 99.9% but the model is useless. Why?

Precision and Recall

- **Precision:**
 - The proportion of true positive predictions out of all positive predictions made:
 - Example: how many predicted fraud cases are actually fraud).
 - Important when false positives are costly (e.g., flagging legitimate transactions as fraud).

$$\text{Precision} = \frac{TP}{TP + FP}$$

Precision and Recall

- **Recall:**
 - The proportion of actual positives that the model correctly identified
 - Example: how many real fraud cases the model catches.
 - Important when missing real cases has serious consequences (e.g., missing actual fraud cases).

$$\text{Recall} = \frac{TP}{TP + FN}$$

Pytorch

What is Pytorch?



- **Open-source deep learning framework** developed by Meta AI (Facebook) in September 2016
- Built on **Torch**, a scientific computing framework (which came out in 2002, based on Lua)

Primary Goal of Pytorch?



- **PyTorch was specifically designed for building and training neural networks (which are used by our models)**
- **PyTorch is not a neural network itself!**

How is PyTorch used?



1. Research & Prototyping

- Quick experimentation with models

2. Model Training

- GPU/TPU acceleration via `torch.cuda`

3. Inference at Scale

- TorchScript or ONNX export

4. Deployment

- APIs with TorchServe
- Mobile with PyTorch Mobile

What's Pytorch commonly used for?



🔍 What Is PyTorch Commonly Used For?

Area	PyTorch Usage
🧠 LLMs & NLP	✅ Hugely popular (BERT, GPT, etc.)
🖼️ Computer Vision (CV)	✅ Still widely used (ResNet, YOLO, ViT)
🔊 Speech & Audio	✅ Strong support (Wav2Vec, Tacotron2)
📖 Academic Research	✅ Preferred framework
🏢 Production ML/AI	✅ Supported via TorchScript, TorchServe, etc.

GPT (LLM) and Pytorch?



GPT and PyTorch: The Connection

GPT Version	Framework Used	Notes
GPT-1 (2018)	PyTorch	Research code was released publicly in PyTorch
GPT-2 (2019)	PyTorch	OpenAI released weights + PyTorch code
GPT-3 (2020)	PyTorch (likely)	Code not released, but OpenAI used PyTorch internally
GPT-4 (2023)	Unknown	Architecture and training details not public
GPT-3.5 Turbo / ChatGPT	Mixed/Proprietary	Likely transitioned to Triton , custom infra, multi-framework stack

How Pytorch uses neural networks



- Pytorch provides tools to build, train, and run neural networks
- Using tensors (multi-dimensional data)

What about TensorFlow?



- Released by Google in 2015. Open Source
- Goal: give researchers a flexible production ready platform for Machine Learning
- Most models can be converted from Pytorch to TensorFlow (via weight conversion)

Why is PyTorch preferred (especially in research)?



- **PyTorch** uses *dynamic computation graphs* (define-by-run), which feel like regular Python code.
- **TensorFlow 1.x** used *static graphs*, which made debugging and iteration more difficult.
- Researchers and students preferred PyTorch because they could **experiment and debug faster**.

💬 “It just feels more like Python” — a common reason cited by developers.

What is a tensor?



- A **tensor** is basically a **multi-dimensional array** — kind of like a generalization of numbers, vectors, and matrices.
- It's the basic building block of data in machine learning — whether it's an image, a sentence, or a time series.

What is a multidimensional array?



- 1D Array - just a list of numbers - a single line:
 - [1, 2, 3]
 - Not “Multi” dimensional since it’s just 1 dimension!
- 2D Array - rows and columns like a spreadsheet:
 - [
 - [1, 2, 3],
 - [4, 5, 6]
 -]

What is a multidimensional array?



- 3D Array - like **pages of 2D matrices** stacked on top of each other — like a book:

```
[  
  [  
    [1, 2, 3],  
    [4, 5, 6]  
  ],  
  [  
    [7, 8, 9],  
    [10, 11, 12]  
  ]  
]
```

Number of dimensions == RANK



- The rank of dimensions is aka RANK - here RANK is 3:

```
[  
  [  
    [1, 2, 3],  
    [4, 5, 6]  
  ],  
  [  
    [7, 8, 9],  
    [10, 11, 12]  
  ]  
]
```

Most ML tasks use 4 to 5 dimensions



- The rank of dimensions is aka RANK - here RANK is 3:

```
[  
  [  
    [1, 2, 3],  
    [4, 5, 6]  
  ],  
  [  
    [7, 8, 9],  
    [10, 11, 12]  
  ]  
]
```

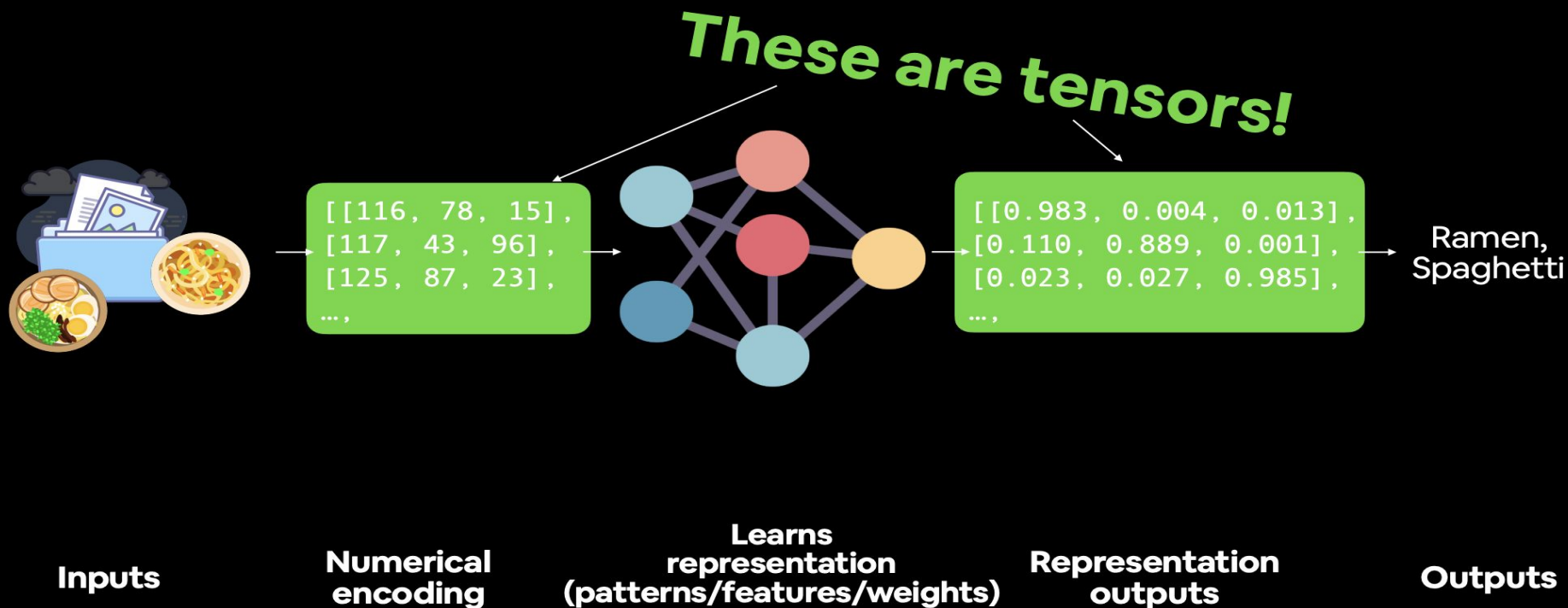
How many dimensions does a tensor have?

Most machine learning tasks use **0D to 5D tensors**, depending on the type of data. There is no limit in TensorFlow or Pytorch

Common Examples:

Tensor Rank	Dimensions	Shape Example	What It Represents
0D	Scalar	<code>()</code>	A single number: <code>7.0</code>
1D	Vector	<code>(3,)</code>	<code>[1.0, 2.0, 3.0]</code>
2D	Matrix	<code>(2, 3)</code>	Table/grid: 2 rows × 3 cols
3D	Cube	<code>(4, 2, 3)</code>	Stack of 2D matrices
4D	Batch of images	<code>(32, 3, 224, 224)</code>	32 RGB images, 224×224 pixels
5D	Videos or sequences of images	<code>(10, 32, 3, 224, 224)</code>	10 videos, each with 32 frames

What is a tensor?



What is TensorFlow?



- Able to access many pre-built deep learning models (TensorFlow Hub)
- Whole stack: preprocess data, model data, deploy model in your application •
- Originally designed and used in-house by Google (now opensource)

Why are tensors important?



- The rank of dimensions is aka RANK - here RANK is 3:

```
[  
  [  
    [1, 2, 3],  
    [4, 5, 6]  
  ],  
  [  
    [7, 8, 9],  
    [10, 11, 12]  
  ]  
]
```

Tensors and neural networks?



- TensorFlow is not a neural network itself, but rather a framework for building and training a neural network and other models

TensorFlow vs Pytorch



- Both TensorFlow and Pytorch are used to create models
 - Generally TensorFlow is considered to be more "production-ready" out of the box
 - Which is likely why companies like Salesforce uses it - it focuses on large-scale production deployments, scalability, etc..
 - Think of it like this: TensorFlow is more "go to market" ready than Pytorch, which is more "Pythonic" - ie pure data science!
 - A lot of models have been created by PyTorch, but we can still work with them in TensorFlow b/c they have weight conversion methods

TensorFlow Serving vs Flask?



- Both TensorFlow Serving and Flask can serve ML models
- However, Flask is much more lightweight compared to TF Serving and is NOT optimized for serving ML models

Is TensorFlow only used for LLMs?



TensorFlow

No, beyond LLM's TensorFlow is commonly used for:

- Image processing
- Audio & Speech
- NLP

Why Tensorflow?



TensorFlow



Easy model building

Build and train ML models easily using intuitive high-level APIs like Keras with eager execution, which makes for immediate model iteration and easy debugging.



Robust ML production anywhere

Easily train and deploy models in the cloud, on-prem, in the browser, or on-device no matter what language you use.



Powerful experimentation for research

A simple and flexible architecture to take new ideas from concept to code, to state-of-the-art models, and to publication faster.

Why Tensorflow?



François Chollet ✓

@fchollet

...

With tools like Colab, Keras, and TensorFlow, virtually anyone can solve in a day, with no initial investment, problems that would have required an engineering team working for a quarter and \$20k in hardware in 2014

7:03 AM · Nov 21, 2020 · Twitter for Android

Why use TensorFlow Serving?



TensorFlow

- When we want to **serve machine learning models in production reliably, efficiently, and at scale**
- Designed for **low-latency inference** — important for real-time predictions (e.g. fraud detection, chatbots).
- Works well in **cloud deployments** (e.g. Kubernetes, ECS).
- You do NOT have to write Flask routes
 - Just save your model as a **SavedModel** and serve it directly. Boom — done.
- REST **and** gRPC support out of the box

TensorFlow Serving vs Flask?

Feature	TensorFlow Serving	Flask
Purpose	Serve machine learning models (especially TensorFlow models) in production	General-purpose web framework (can serve ML models, but not optimized for it)
Out of the Box	Model loading, versioning, batching, high-performance inference	You write all logic manually — model loading, request parsing, inference, etc.
Protocols	REST and gRPC	REST (by default)
Performance	Highly optimized, C++ backend, faster inference	Python-based, slower for high-load or real-time serving
Model Format	Requires <code>SavedModel</code> format	Can serve any model as long as you code it (TensorFlow, PyTorch, etc.)
Scalability	Production-grade: easy to scale with Kubernetes, load balancers, etc.	Requires additional layers to scale well (e.g. uWSGI, Gunicorn, Docker orchestration)
Versioning Support	✅ Built-in	❌ You'd have to code it manually
Batching Support	✅ Automatic batch requests for better performance	❌ Manual work needed
Live Model Reloading	✅ Can detect new versions in model directory	❌ You must restart the app or add reload logic



TensorFlow

Google Colab Overview

But first...Why do data scientists use Python?

Readable Syntax: Python's syntax is clean and intuitive, which makes it accessible for beginners and allows experienced programmers to write clear and maintainable code.

Rich Ecosystem of Libraries and Frameworks: Libraries like NumPy provide powerful tools for handling arrays and performing high-performance mathematical operations.

The **Pandas library** offers flexible data structures and functions for manipulating structured data, making tasks like cleaning, transforming, and analyzing data straightforward.

Strong Community and Support: The community contributes to the ongoing development of libraries and tools

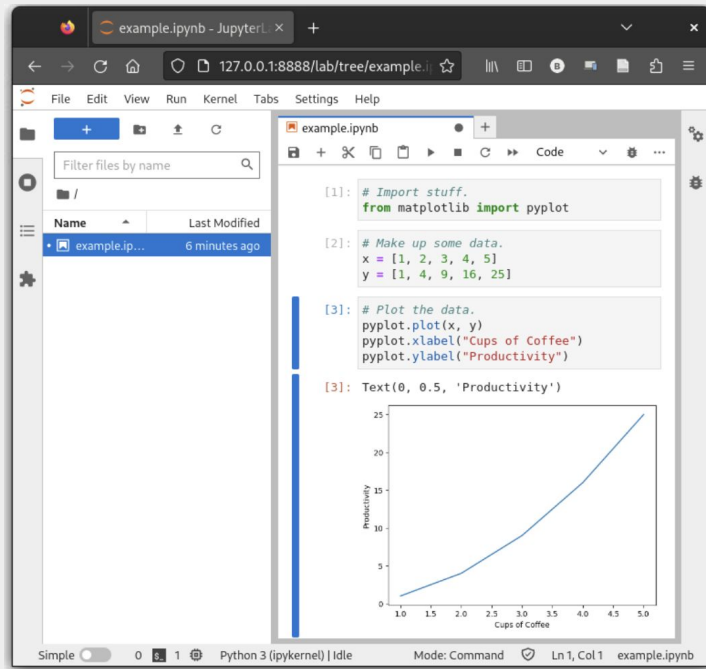
Open Source Nature: Being an open-source language, Python is free to use. This reduces barriers for startups, researchers, and educational institutions.

But first...What is a Jupyter notebook?

Jupyter Notebook is a web-based tool for writing and running code, mainly in Python.

It's great for data analysis, machine learning, and sharing work with others.

With live code, visuals, and text in one place, it makes coding more interactive and easy to use.

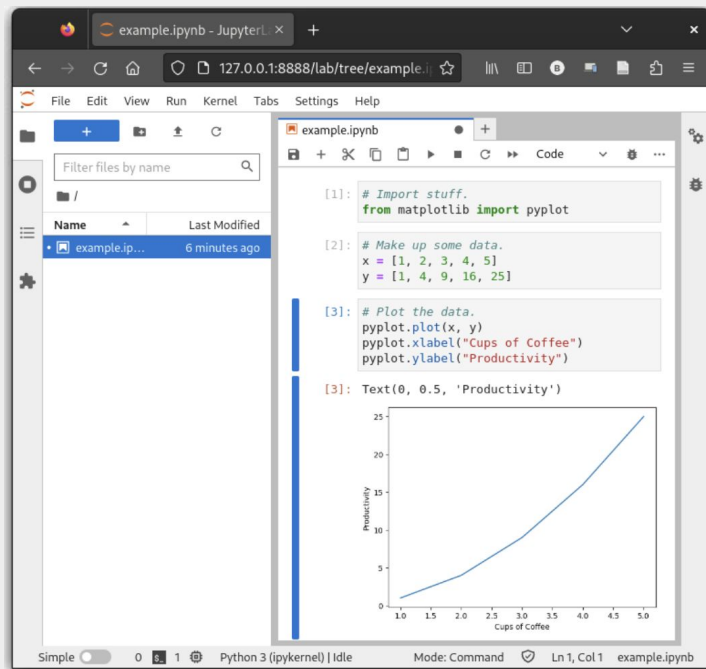


But first...What is a Jupyter notebook?

Jupyter Notebooks have a .ipynb extension

ipynb == interactive Python notebook

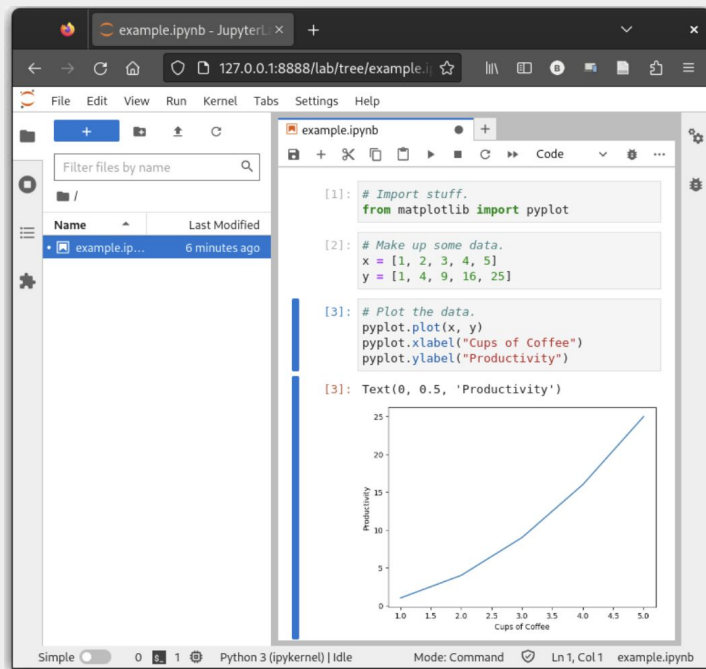
Under the hood of a Jupyter notebook it's all JSON



Google Colab

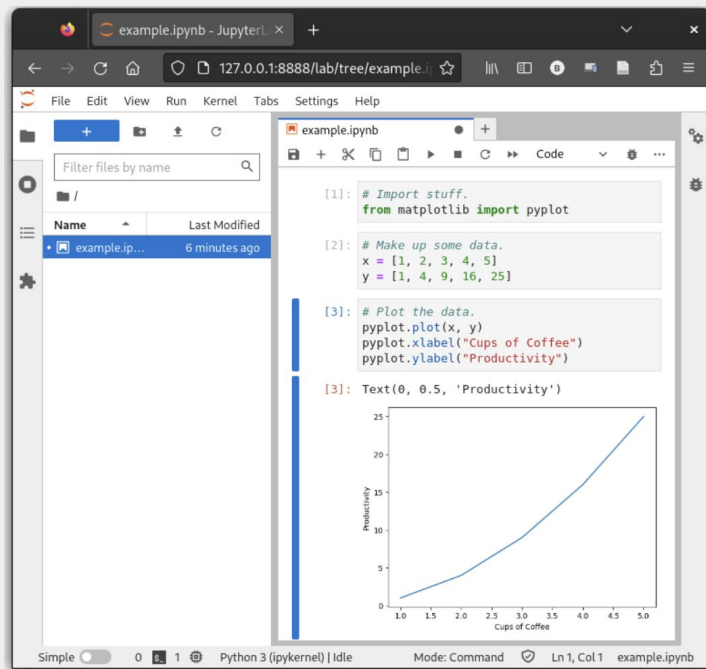
Google Colab is a cloud-based Jupyter notebook environment that allows users to write and execute Python code with free access to GPUs and TPUs.

It is widely used for machine learning, data analysis, and collaborative coding without requiring local setup.



Google Colab

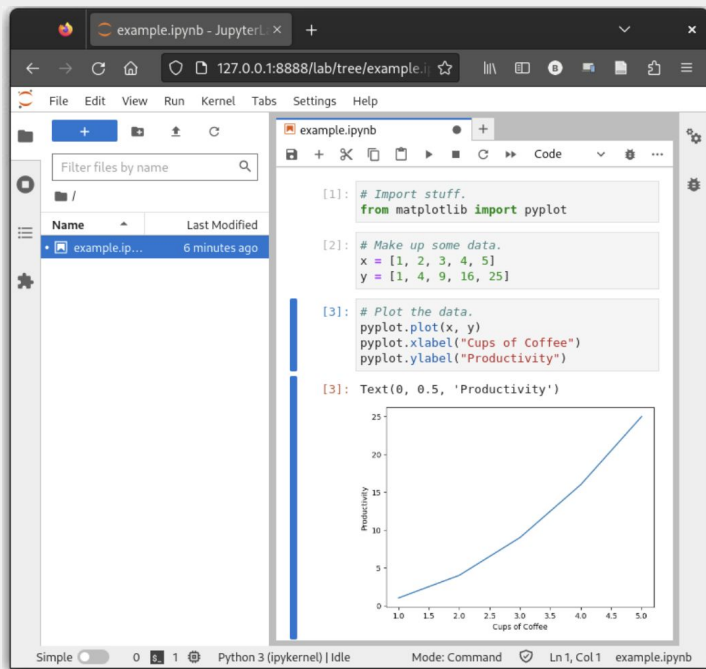
**When Google Colab executes code
where exactly is it running that code?**



Google Colab

When Google Colab executes code where exactly is it running that code?

On a Google compute in the cloud (like an EC2 in AWS). Google gives some compute power for free. The pro version gives you even more

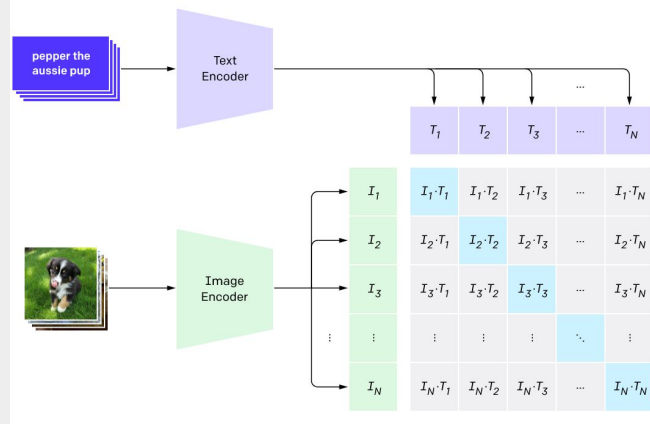


Is Colab only for Jupyter notebooks?

Google Colab is a cloud-based Jupyter notebook environment that allows users to write and execute Python code with free access to GPUs and TPUs.

It is widely used for machine learning, data analysis, and collaborative coding without requiring local setup.

1. Contrastive pre-training



Colab Pro vs free account

Pro gives you more compute - CPU and memory

Runtime disconnected

The runtime disconnected due to inactivity. As a Colab Pro subscriber your idle timeouts are more lenient than they are for non-subscribers, but runtime durations are still not guaranteed or unlimited. [Learn more](#)

If you are interested in even longer runtimes with even more lenient timeouts, you may want to check out [Colab Pro+](#).

Close

Reconnect

Keras

Keras and TensorFlow

Keras is a high level API meant to work with TensorFlow

It hides a lot of the complex stuff like:

- Model definition
- Training loops
- Optimizers
- Loss calculation
- Metrics



Keras and TensorFlow



TensorFlow

- Keras was originally a standalone library, but now it's fully integrated into **TensorFlow**
- Currently it only works with TensorFlow (older versions of Keras may be able to work with other backends)

Keras and TensorFlow



TensorFlow

- So instead of writing 50+ lines of boilerplate with just TensorFlow, with Keras you can write a few lines of code
- If you skip Keras and go with **pure TensorFlow**, you're working closer to the "metal":
- That gives you **more control**, but it's also **more code** and **easier to mess up**.
- Unlike TensorFlow, **PyTorch is more user-friendly by default** — even its lower-level API feels intuitive. (Pytorch does still have libraries like Keras - like lightning)...

Keras/TensorFlow analogy

Keras is to TensorFlow like **React is to the DOM**.



- Keras gives you high-level components to build models easily.
- React gives you high-level components to build UIs easily.

TensorFlow does the math just like **the DOM actually renders stuff**.

Without Keras, you're writing raw TensorFlow — similar to writing raw DOM manipulation in JavaScript. Doable, but much more verbose.

DEEP LEARNING PRIMER

Tensors

A **tensor** (in the context of deep learning) is a multi-dimensional array of numbers. Tensors are described by the number of **dimensions** (how many levels of nesting) they have, and the **size** of each dimension, in order from outermost to innermost.

Tensors are **dense**, meaning that all slots contain values.

Scalar

0D: 1 (size [])

Vector

1D: [1, 2, 3] (size [3])

Matrix

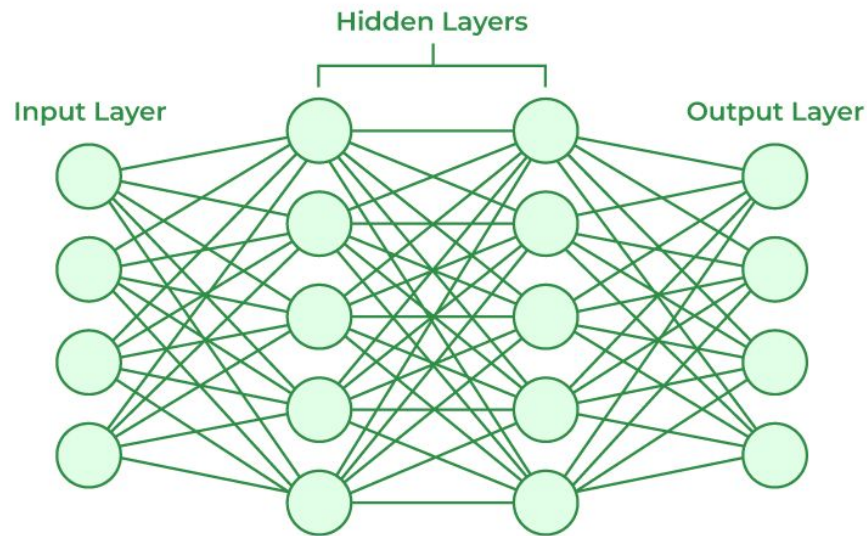
2D: [[1, 2, 3],
[4, 5, 6]] (size [2, 3])

N-D Tensor

3D: [[[1, 2, 3],
[4, 5, 6]],
[[7, 8, 9],
[10, 11, 12]]]

Layers

Neural networks are built out of multiple **layers**. There are many types of layers, each with various properties that make them ideal for different tasks. Layers are characterized by the number of **inputs** and **outputs** they have (referred to as neurons) and how inputs influence outputs (referred to as **weights** or **parameters**).

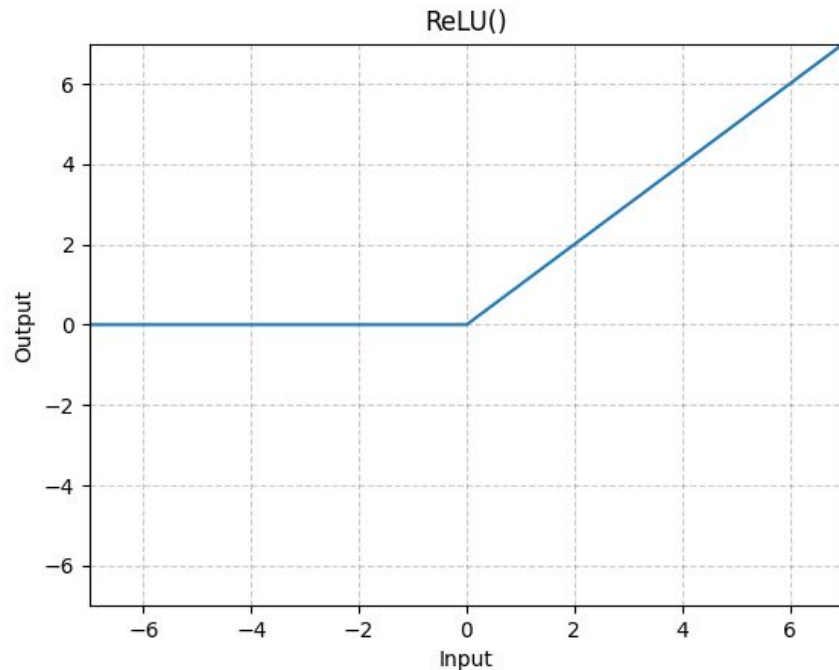


[Image credit](#)

Activations

Between layers there is almost always an **activation function**. These are a **non-linear** mapping of inputs to some output range. Since layers apply **linear** transformations to their input, activations allow for learning non-linear functions.

Activations work **element-wise** on all outputs from the previous layer.

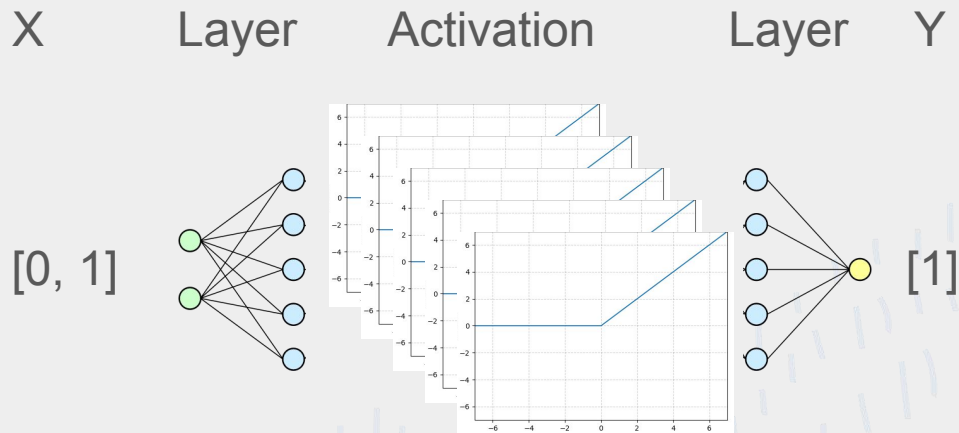


[Image credit](#)

Neural Networks

A **neural network** (minimally):

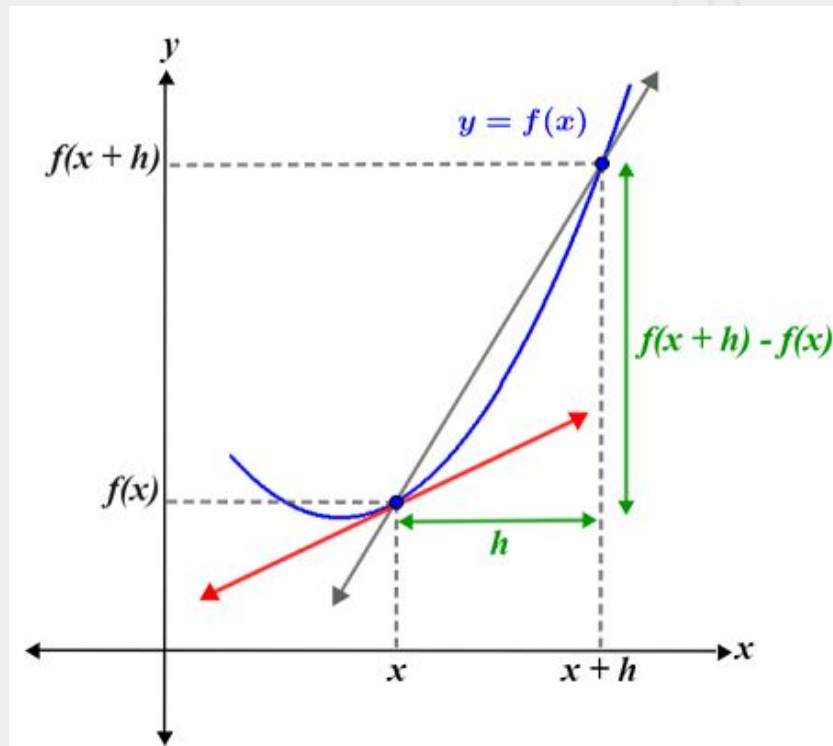
- Is a stack of **layers** with **activations** between them
- Learns a (**non-linear**) mapping between inputs and outputs
- Takes a **tensor** as an input
- Produces a **tensor** as an output



Learning

Models like neural networks learn through a process called **optimization**. Specifically, neural networks are **differentiable** and thus can be optimized by **gradient descent**. This happens by:

1. Passing inputs through the model
2. Comparing the output to the target
3. Finding the difference (called **loss**)
4. Updating model weights to reduce the loss next time



[Image credit](#)

Objective/Loss Functions

In order to compute the loss (the difference between the predicted output and the target) we use an **objective function** (also called a loss function). This allows us to compute a number (lower is better) that we use to:

- Measure how well the model is working
- Compute the gradient for updating the model weights

$$\ell(x, y) = L = \{l_1, \dots, l_N\}^\top, \quad l_n = - \sum_{c=1}^C w_c \log \frac{\exp(x_{n,c})}{\sum_{i=1}^C \exp(x_{n,i})} y_{n,c}$$

[Image credit](#)

Objective/Loss Functions: General

Discrete Output

- Softmax + Cross Entropy
 - Output is a probability distribution
 - Target class probability increases, all others decrease
- Typically used for
 - Language Models
 - Models with sequential output

Continuous Output

- Mean Squared Error
 - Output is a regression
 - Estimate the average value
- Typically used for
 - Image generation
 - Models with parallel output

Aside: Discrete vs. Continuous Output

Discrete

- Basically a classification problem
- All possibilities are independent and... discrete
- Model outputs the probability of a known possibility

Continuous

- Basically a regression problem
- All possibilities continuous, real numbers
- Model outputs the estimated value directly

Objective Functions: Auxiliary

Similarity Constraints

- Encourage Clustering based on some known criteria
 - Connection/relationship between elements
 - Can be in data space or latent space
- Self or cross similarity matrix + MSE
 - Standard supervised learning

Secondary Model

- Text to Image (e.g. [CLIP](#))
 - Encode input text
 - Encode output image
 - Compare similarity
- Reward model (e.g. RLHF)
 - Use a model to “rate” the output of your model

just to name a few

OVERVIEW OF GENERATIVE AI

Notable Examples

Some of the most well known examples are:

- Text generation (e.g. ChatGPT, Claude, Gemini, LLaMA, Github CoPilot)
- Text to image (e.g. DALL-E, Stable Diffusion, Midjourney)
- Text to Video (e.g. Runway, Pika, Sora)
- Text to music (e.g. MusicLM, Riffusion, Suno, Udio)



Text to Image

Feb 2022 (V1) to Dec 2023 (V6)

Midjourney Version Comparison



<https://www.linkedin.com/pulse/midjourney-v6-released-algie-desucatan-rdvxc>

Text to Video

2023



2024



Text to Music

2020

[Amper Music](#)

2022

[Riffusion](#)

2024

[Google MusicFX](#)

[Suno](#)

[Udio](#)

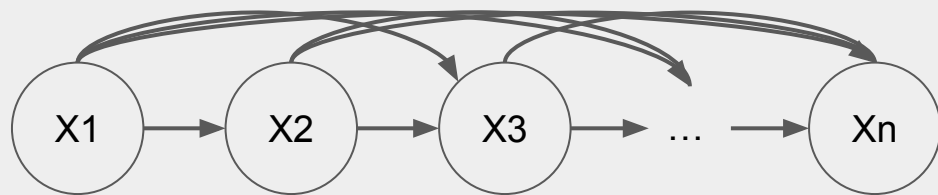
TYPES OF GENERATIVE MODELS

Autoregressive Models

Autoregressive models use an input sequence to predict the next step, then feed the output back into the model to make the next prediction, and so on.

Use Cases:

- Autocomplete on your phone
- Sequence generation

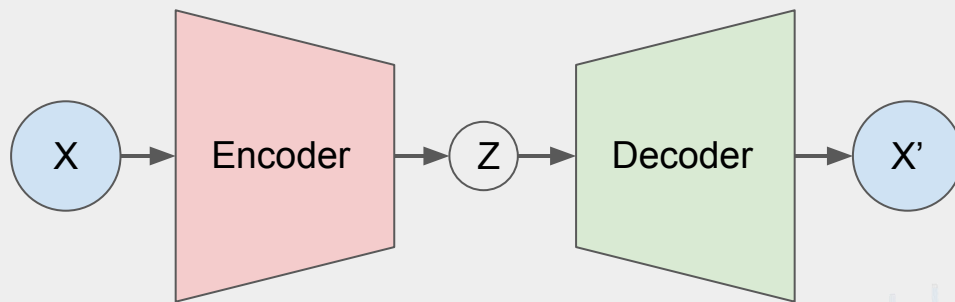


Autoencoders

Autoencoders are trained to take an input, compress it, and reconstruct the original.

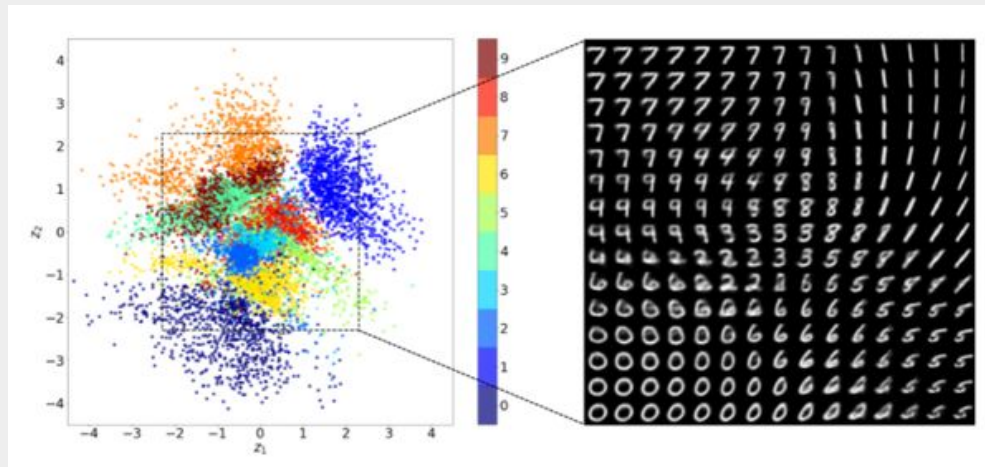
Use Cases:

- Compression
- Clustering
- Data generation



Variational Autoencoders (VAEs)

A notable flavor is the Variational Autoencoder (VAE), which can learn to smoothly transition between examples.

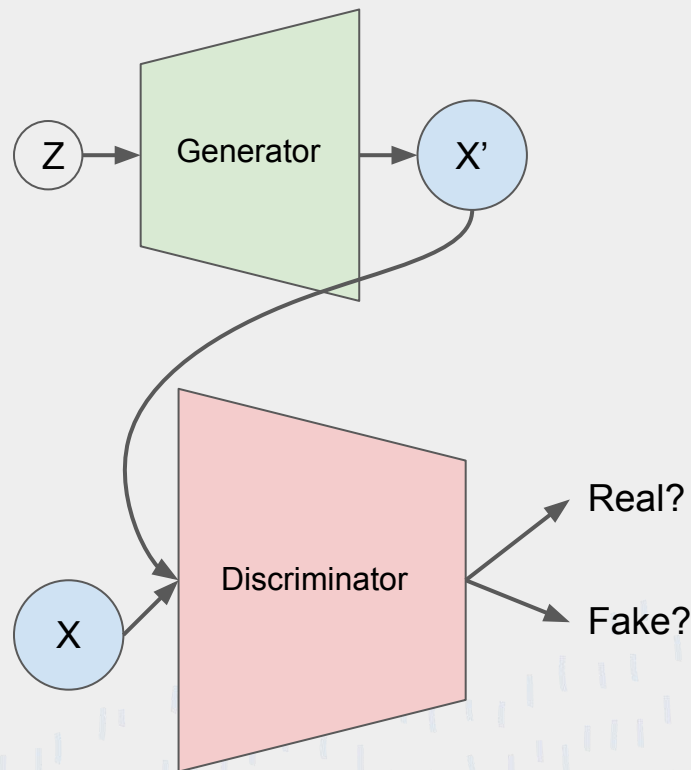


Generative Adversarial Networks (GANs)

Generative Adversarial Networks (GANs) are a pair of networks playing a game. The Discriminator looks at inputs and predicts if they are real data or fake. The Generator makes fake data and tries to trick the Discriminator.

Use Cases:

- Data generation
- Data editing



Generative Adversarial Networks (GANs)

GANs have been used extensively in image generation, and image editing.

e.g. [This X Does Not Exist](#)

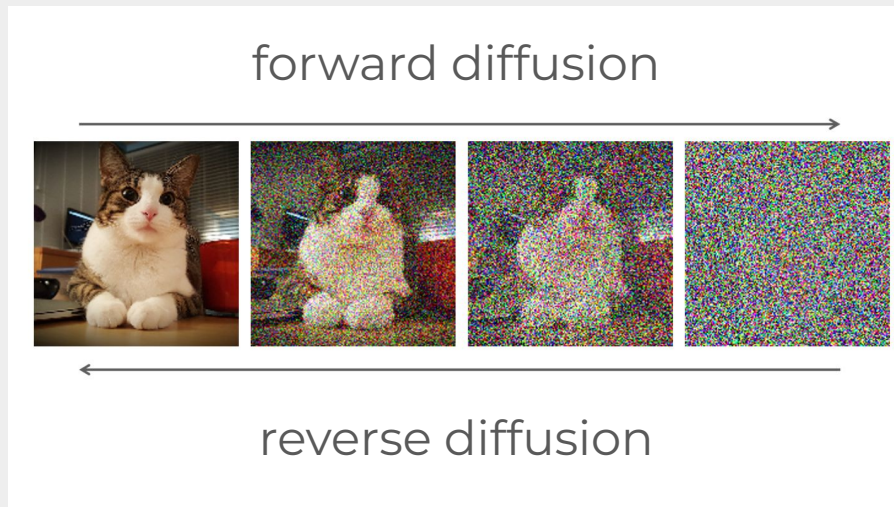


Diffusion Models

Diffusion Models learn to gradually remove noise from data to produce their output.

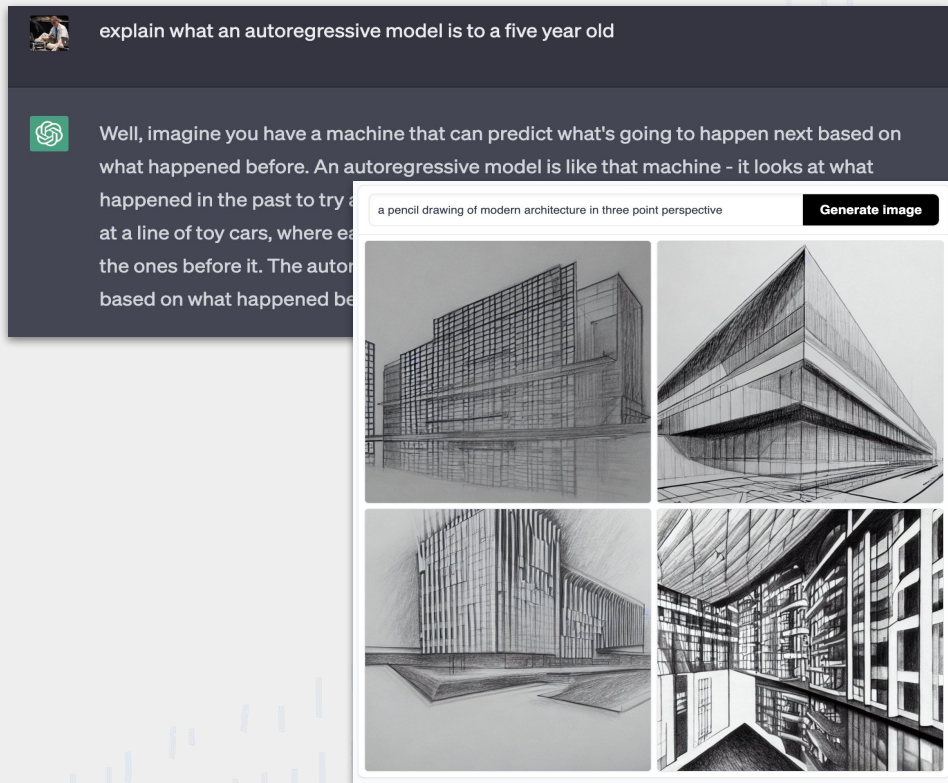
Use Cases:

- Data generation
- Data editing



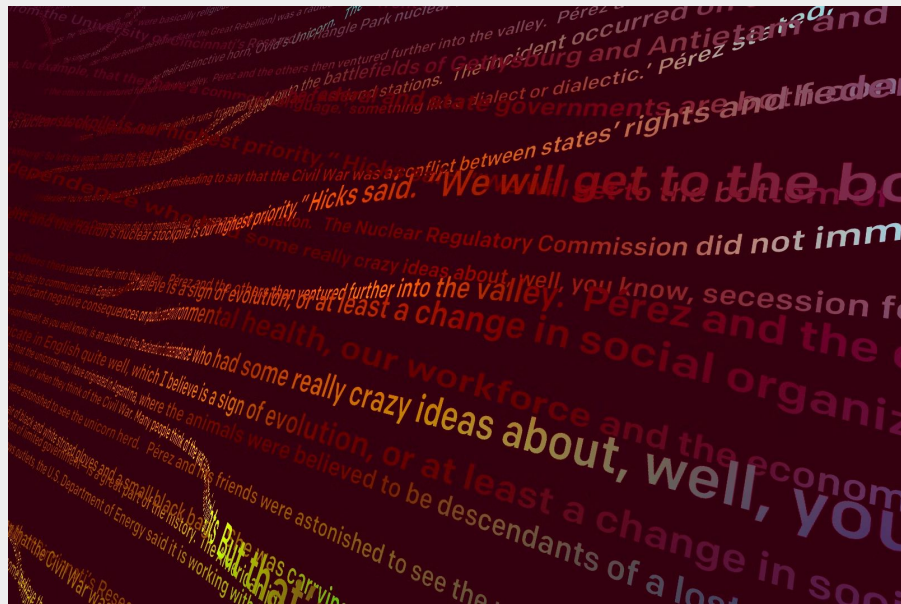
The raise of prompting

Today's most well known models revolve around **prompting**: using text instructions to guide their output.



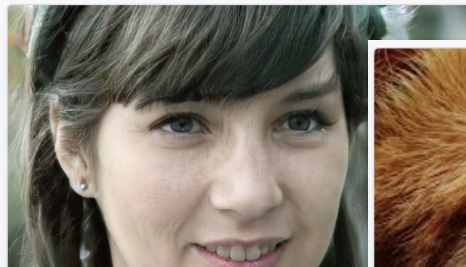
The raise of prompting

Autoregressive language models like **GPT-2** are trained to produce continuations of text provided as their input. The significantly larger **GPT-3** improved these capabilities, and later the **InstructGPT** framework led to the current conversational paradigm we see with **ChatGPT**.



Gathering Training Data - Image Generation

Image generation models are trained to take an input of some kind and output an image. For example “this cat does not exist” (a GAN) is trained to turn noise into realistic photos of cats by training on a significant number of photos of cats.



This Person Does Not Exist

The site that started it all, with the that says it all. Created using a style based generator (StyleGAN) by a community of people who are intrigued and

Created by



This Cat Does Not Exist

made cats will and make you through your screen in a while the cats due to model – beware, they

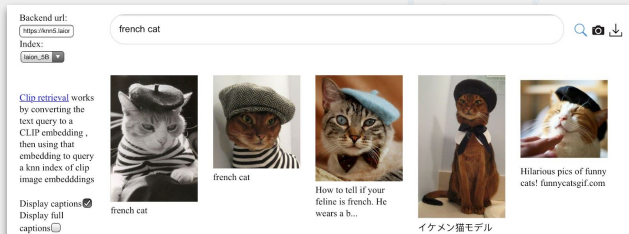


This Rental Does Not Exist

Why bother trying to look for the perfect home when you can create one instead? Just find a listing you like, buy some land, build it, and then enjoy the rest of your

Gathering Training Data - Text to Image

Text to Image models are trained on large quantities of (text, image) pairs (captions, alttext, hand labeling). They rely on the correlation between aspects of the text and images across the dataset.



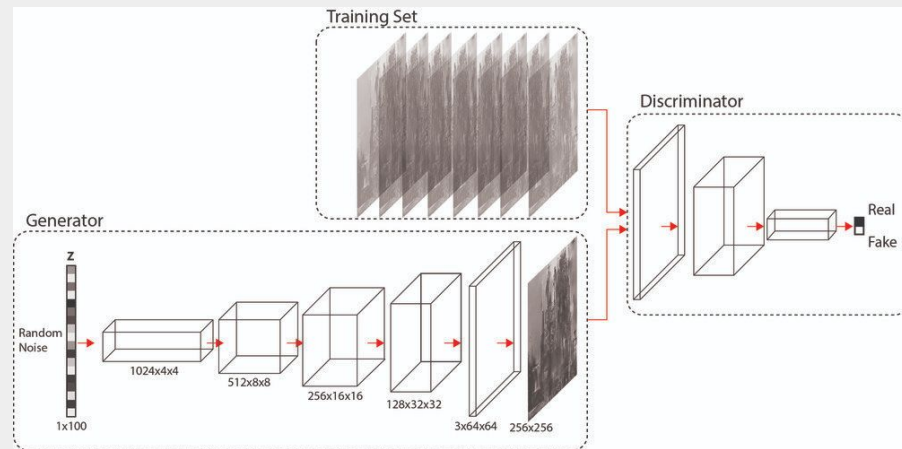
What was the Stable Diffusion model trained on?

The underlying dataset for Stable Diffusion was the 2b English language label subset of LAION 5b <https://laion.ai/blog/laion-5b/>, a general crawl of the internet created by the German charity LAION.

Choosing a Model: Image Generation

Image generation models operate on **continuous** inputs and outputs. Inputs are typically either random “noise”, or the output of some encoder. This is typically the realm of:

- Variational Autoencoders
- Generative Adversarial Networks
- Denoising Diffusion Models



(a typical GAN architecture)

Choosing a Model: Image Generation

VAE

- Encoder/Decoder pair
- Trained to
 - Reconstruct input
 - Form normally distributed clusters

GAN

- Generator creates fake images from “noise”
- Discriminator tries to distinguish fake from real
- Discriminator gradient is used to train Generator

Diffusion

- Typically a UNet (image to image model)
- Iteratively add noise to an image
- Train a model to predict the noise to remove at a given step

Choosing a Model: Image Generation

VAE

- Encoder is useful for finding a latent vector to start from
- Latent space is nicely clustered and smooth
- Results are often blurry or overly “sterile”

GAN

- Very realistic output
- Lack of encoder makes latent space difficult to navigate
- Difficult to train

Diffusion

- Able to start from noise or an existing image
- Easy and stable to train
- Tend to have issues with quantities of repeated elements (hands, arms, fingers, etc.).

Choosing a Model: Image Generation

Autoregressive

The decoder/generator produces its output one pixel at a time, using all previously generated pixels as its context.

Deconvolutional

The decoder/generator progressively increases the size of its output, pushing information in the channel dimension into the spatial dimensions.

This is typically the preference.

Choosing a Model: Image Generation - Inputs

Models typically work on a fixed image size (e.g. 512x512) with RGB values in the 0.0 - 1.0 range. Larger inputs can be cropped. Smaller images can be upscaled, although this reduces quality.

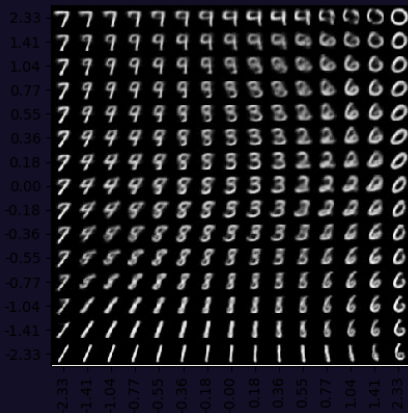
Choosing a Model: Image Generation - Outputs

The final output of the model is an RGB image with the width and height it was trained on, with all colors in the 0.0 - 1.0 range.

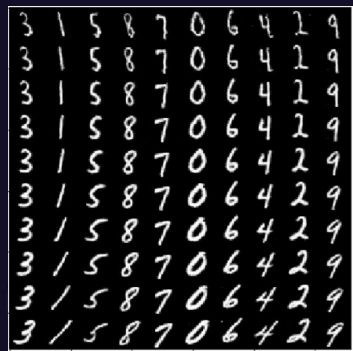
Aside: Conditional Generation

Generative models can be made to take a **condition** input to dictate what class its outputs should be from. For example, the decoder of a VAE trained on MNIST could take the **class of a desired digit** as an input, and would produce variations of that particular digit.

Base

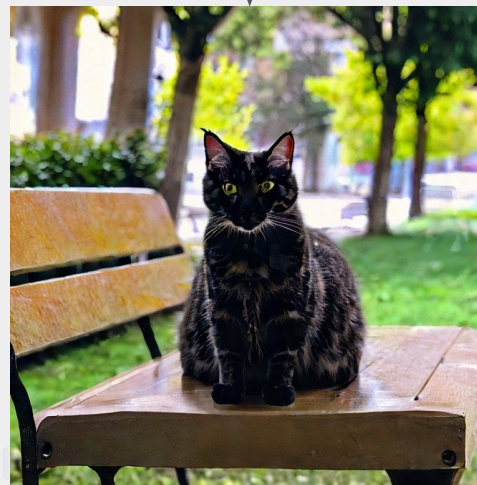
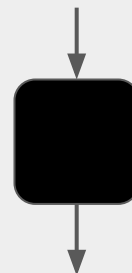


Conditional



Choosing a Model: Text to Image

“a cat sitting on a bench in the park”

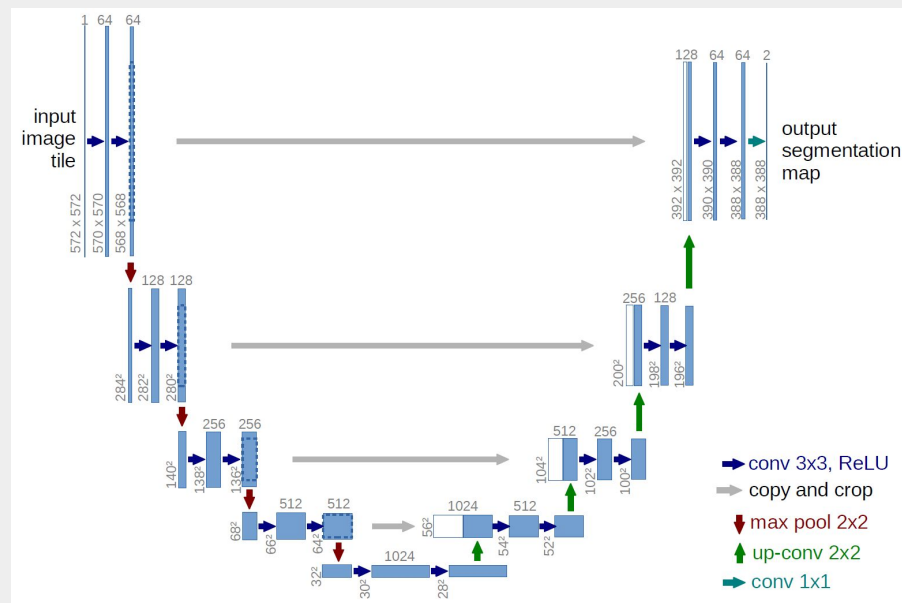


Text to image models can be thought of as image generation models with a **condition input** that is the output of a **text embedding model**.

Choosing a Model: Text to Image

In theory, any of the architectures we have talked about so far can work. However, the current generation of text to image models (DALL-E 3, Midjourney, StableDiffusion) are all:

- Transformers for text embedding
- Diffusion models (UNet) for image generation



Choosing a Model: Text to Image - Input / Output

It should be no surprise that inputs and outputs for these models are handled in the exact same way as their previous counterparts.

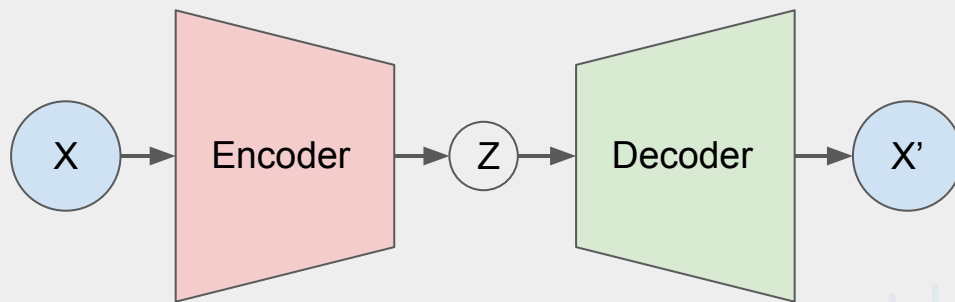
BUILDING GENERATIVE MODELS

Review: Autoencoders

Autoencoders are trained to take an input, compress it, and reconstruct the original.

Use Cases:

- Compression
- Clustering
- Data generation



OVERVIEW OF NLP: Understanding Language as Data

NLP



Speech recognition



Text analysis



Translation



Language
generation

Machine Learning with NLP



Supervised Learning:

Sentiment Analysis
Spam Detection
Intent Detection



Unsupervised Learning:

News Article Clustering
Customer Feedback Clustering
Document Organization

All ML falls into one or more of these categories:

Category	Description	Example
✓ Supervised Learning	Train on labeled data ($X \rightarrow y$)	Spam detection, fraud detection
✓ Unsupervised Learning	Find patterns in unlabeled data	Customer segmentation, anomaly detection
✓ Semi-supervised Learning	Mix of labeled and unlabeled data	Label-efficient classification (e.g., medical imaging)
✓ Self-supervised Learning	Predict part of data from other parts (labels are <i>created</i> from the data itself)	BERT (masked token prediction), SimCLR
✓ Reinforcement Learning	Learn by interacting with environment and receiving rewards	Robotics, game-playing (e.g., AlphaGo)

NLP Labels and Features

Label:



Sentiment
News article type
Intent

Features:



Text is unstructured
ML models require numerical features

A Brief History of NLP



1950s:

Early Concepts
Turing Test
Chomsky's theory of syntax.



1960s - 1980s:

Rule-based systems
ELIZA and SHRDLU
Statistical approaches to language processing



2000s:

Advent of machine learning in NLP
Support Vector Machines (SVM)
Hidden Markov Models (HMMs).



2010s:

Major progress in deep learning and the introduction of neural networks.



2017:

The introduction of Transformers revolutionized NLP with models like BERT and GPT.

A Brief History of NLP

1950s–1980s: Early Rule-Based Systems

- **1954:** Georgetown-IBM experiment translates 60 Russian sentences to English. Hype begins.
- **1960s–70s:** Hand-coded **grammar-based** systems (e.g., SHRDLU, ELIZA) for basic dialogue.
- **1980s:** Still rule-heavy. Used in **defense, government, and limited commercial tools** (e.g., parsing, info retrieval in intelligence).

History - shift to probabilistic instead of rule-based

1990s: Statistical NLP Takes Over

- Shift to probabilistic models — big moment!
- Practical Use:
 - Search engines (early Google, AltaVista) use NLP for ranking and stemming.
 - Spell check, grammar check tools (MS Word) emerge.
 - Information extraction (e.g., names from news articles) in finance and law.

NLP History

2000s: NLP in the Real World

- NLP enters mass-market products:
 - **Spam filters** (Naive Bayes, SVMs)
 - **Text classification** for support tickets, news, legal docs
 - **Sentiment analysis** in marketing
- **Named Entity Recognition (NER)** and **part-of-speech tagging** become standard
- Start of **machine translation** at scale (Statistical MT, e.g., Moses)

NLP History

📱 2010s: Deep Learning Revolution

- 🔥 **2013:** Word2Vec by Google → turns words into vectors; powers early **chatbots** and **recommendation systems**.
- 🔥 **2015–2017:** LSTM and attention-based models dominate:
 - **Chatbots** in banks, e-commerce
 - **Voice assistants:** Siri, Alexa, Google Assistant
 - **Neural Machine Translation** replaces statistical MT (Google Translate ~2016)
- 🔥 **2018:** **BERT** from Google — deep bidirectional understanding explodes across NLP.

NLP History

2020s: Foundation Models & GenAI

- **Transformers** and **self-supervised pretraining** dominate:
 - **GPT-3 (2020)**, **ChatGPT (2022)** make NLP *interactive* and *open to the public*
 - Enterprise uses: legal summarization, coding assistants (e.g., Copilot), customer service agents, knowledge management
- Practical NLP everywhere:
 - Email autocomplete (Gmail)
 - E-commerce search (Amazon, Walmart)
 - Enterprise AI copilots (Microsoft 365, Salesforce)

NLP History

📌 2025 and beyond?

- **Multimodal NLP** (text + image + audio)
 - **Secure NLP**: provenance, watermarking, hallucination control
 - **On-device NLP**: running language models at the edge
 - **Smaller, distilled models** for niche applications (mobile, offline)
-

Why the shift to probabilistic?

Why the Shift to Probabilistic Models in NLP Happened

Problems with Rule-Based Systems:

1. **Manual labor:** Every rule had to be hand-coded by linguists — not scalable.
2. **Brittle:** Rules broke easily with variations in language or slang.
3. **Domain-specific:** Rules didn't generalize well across topics or languages.
4. **No ambiguity handling:** Language is full of ambiguity (e.g., "bank"  vs. riverbank ) , and rules struggled to resolve it.

Advantages of probabilistic...

Advantages of Probabilistic (Statistical) Models:

1. **Learn from data:** Models could now learn patterns from **large corpora** without needing explicit rules.
 - e.g., learn that "bank account" is more common than "river bank account"
2. **Handle uncertainty:** Probabilities let models choose the **most likely interpretation** rather than needing perfect grammar.
3. **Scalability:** Easily adapt to new domains by retraining rather than rewriting rules.
4. **Better performance:** Especially in tasks like:
 - Part-of-speech tagging (e.g., Hidden Markov Models)
 - Named entity recognition (NER)
 - Machine translation (Statistical MT)
5. **Explosion of digital text:** The web provided **huge datasets** (news articles, Wikipedia, search logs), making data-driven approaches feasible.

Rules vs statistical summarized

"Writing down the rules of language is hard. Letting the data learn the rules is easier — and works better."

Challenges in NLP

Ambiguity:

- Human languages are ambiguous, with words and phrases having multiple meanings depending on context.

Context Understanding:

- Capturing long-term dependencies in texts.

Limited Real-World Understanding:

- Machines can process language but struggle with true understanding or reasoning like humans.

The Challenge of Representing Text as Data



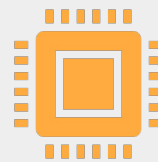
Text is Complex:

inherently ambiguous, nuanced,
and context-dependent
challenging to convert into a
numerical format that computers
can process.



Why It's Difficult:

Words vary in length and
structure.
Meaning depends on word order
and context.
Text data isn't naturally numerical,
but machine learning models
require numerical inputs.



Goal:

Representing text as a numerical
array means converting words and
sentences into structured,
numerical data that algorithms can
process.

Approach: Transform Text to Numbers



Tokenization:

Breaking text into words, phrases, or sentences for analysis.



Stemming and Lemmatization:

Reducing words to their root form (e.g., "running" becomes "run").



Syntax and Parsing:

Analyzing grammatical structure.



Named Entity Recognition (NER):

Identifying key entities like people, places, or organizations.



Word Embeddings:

Representing words in continuous vector space (e.g., Word2Vec, GloVe).

Tokenization

Definition:

Tokenization is the process of breaking down text into smaller pieces (tokens): words, phrases, or even characters.

Why It's Important:

It converts text into a format that machines can work with.
Each token represents a meaningful unit in the text.

Example:

Breaking the sentence
"I love NLP" into tokens:
["I", "love", "NLP"]

Tokenization Approaches

Word-Level Tokenization:

- Splits text into individual words.
- Challenge: Doesn't account for nuances like spelling variations or context.

Sub-word Tokenization (Byte-Pair Encoding, Word-Piece):

- Breaks words into meaningful subunits (e.g., "running" into "run" and "ing").
- Advantage: Reduces vocabulary size, captures morphemes and handles unseen words better.

Character-Level Tokenization:

- Each character is treated as a token.
- Challenge: Too granular, makes it harder to capture semantic meaning.

More Challenges in Tokenization:

- Ambiguity (e.g., homonyms).
- Handling punctuation, numbers, or special characters.

Bag-of-Words Example

- Raw text examples:
 - The cat is on the table
 - The dog is under the table
 - The cat and the dog are friends
- BoW Representation:

and	are	cat	dog	friends	is	on	table	the	under
0	0	1	0	0	1	1	1	2	0
0	0	0	1	0	1	0	1	2	1
1	1	1	1	1	0	0	0	2	0

Traditional Vectorization Techniques

- Bag of Words (BoW):
 - Represents text by counting the frequency of each word in a document, disregarding order.
 - How it works: Create a large matrix where each row is a document and each column is a word. The cell values represent word occurrences.
 - Limitations:
 - Ignores word order and context.
 - High dimensionality: A separate column for every word makes it computationally expensive
- TFIDF (Term Frequency – Inverse Document Frequency):
 - Weighs words by their importance. Common words are downweighted, while rare but informative words are upweighted.
 - How it works: Combines word frequency in a document (TF) with the rarity of the word across documents (IDF).

Introducing Embeddings

Definition:

- Word embeddings are dense vector representations that capture the meaning of words by placing them in a continuous vector space.

Why It's Important:

- Embeddings can capture semantic relationships between words (e.g., "king" and "queen" are closer to each other than "king" and "dog").

Common Embedding Techniques:

- Word2Vec: Predicts a word based on its neighbors (context) to generate a vector space.
- GloVe (Global Vectors for Word Representation): Combines word cooccurrence statistics to create meaningful embeddings.

How Embeddings Work

- Core Idea:
 - Each word is mapped to a point in a vector space, where words with similar meanings are placed closer together.
- The relationship between words can be visualized as vector arithmetic:
 - King : Queen :: Man : Woman
 - "King - Man + Woman $\approx$ Queen"

Advantages of Embedding

Contextual Understanding:

Embeddings can capture semantic and syntactic meanings.



Dimensionality Reduction:

Embeddings reduce the high-dimensional nature of the Bag of Words or TFIDF approach into a low-dimensional, dense representation.



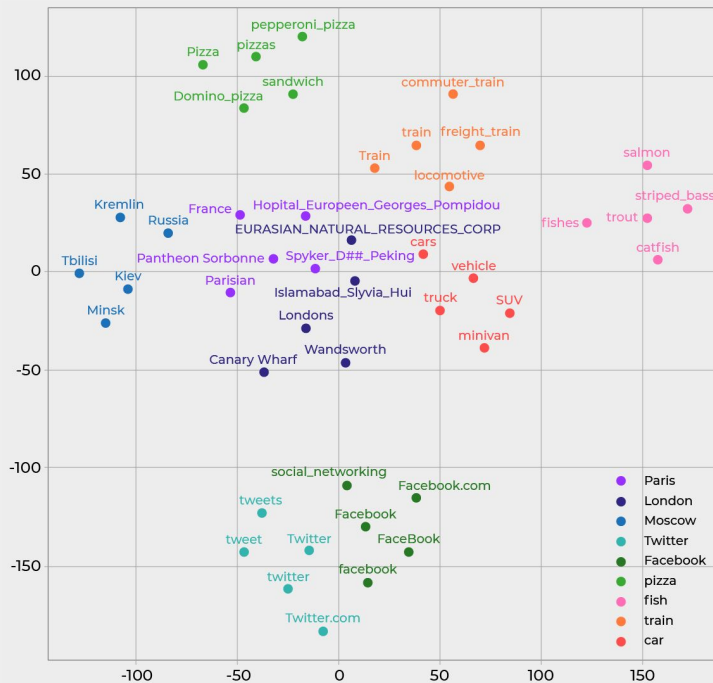
Semantic Relationships:

Embeddings capture relationships like synonyms and analogies.

Embeddings - Word Vectors

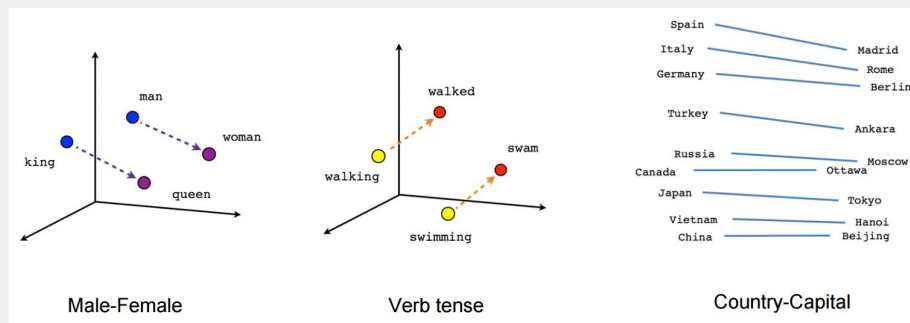
Embeddings represent the **semantic meaning** of tokens. When tokens are words, we refer to them as **word vectors**. Semantic meaning is encoded by how **similar** the vectors are to each other. Similarity means:

- Closeness (distance/angle)
- Clustering



Embeddings - Word Vectors

Word2Vec word embeddings exhibit semantic relationships between vectors as “pseudo-axes” (directions and distances between vectors) that can be used to adjust the meaning of a vector arithmetically. For example: $\text{KING} - \text{MAN} + \text{WOMAN} = \text{QUEEN}$



LARGE LANGUAGE MODELS

Text Generation



Before Deep Learning

N-gram models and Markov chains: based on fixed-size word sequences
Achieved limited success but struggled with coherence over longer texts due to lack of context awareness.



Deep Learning: RNNs

Recurrent Neural Networks (RNNs): Enabled sequential processing but struggled with long-term dependencies due to vanishing gradients.
LSTM (Long Short-Term Memory) and GRU (Gated Recurrent Units): Improved RNNs by allowing models to "remember" long sequences.



Deep Learning: Transformers:

Introduced with the Attention is All You Need paper, replacing sequential processing with attention mechanisms that model relationships between all tokens in a sequence simultaneously

<https://jaroslawwiosna.github.io/markov-chain-text/>

NLP Challenges Prior to Transformers



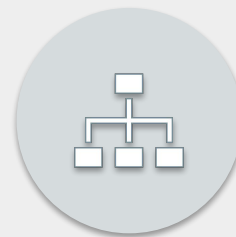
Deep learning struggled to achieve coherent humanlike conversations.



LSTMs cannot capture relationships between distant words in a sequence, leading to a loss of context in long sentences.



RNN-based models processed data sequentially which is inefficient for large datasets.



Pre-transformer models had difficulty understanding complex, multi-layered context.

Gathering Training Data

As with all AI, training data is the life blood of a model, and generative models require:

- Extremely large quantities
- As high quality as you can find
(garbage in, garbage out)



Gathering Training Data - Language Models

GPT-2 was created as a direct scale-up of GPT, with both its parameter count and dataset size increased by a factor of 10.^{[9][10][5]} Both are **unsupervised transformer** models trained to generate text by predicting the next word in a sequence of **tokens**. The GPT-2 model has 1.5 billion parameters, and was trained on a dataset of **8 million web pages**.^[9] While GPT-2 was reinforced on very simple criteria (interpreting a sequence of words in a text sample and

Language models are trained to predict the next token in a sequence (they are autoregressive). Large models require significant amounts of text from the domain you are interested in (conversations, code, research, etc.).

The Pile *An **800GB Dataset** of Diverse Text for Language Modeling*

To summarise:

- GPT-3 is a very large language model (the largest till date) with about 175B parameters.
- It is trained on about **45TB of text data** from different datasets.
- As such the model itself has no knowledge, it is just good at predicting the next word(s) in the sequence. It is not designed to store or retrieve facts.

The Codex model was trained on **tens of millions** of public repositories.

Gathering Training Data - Language Models

Unlike classical NLP, deep learning based models require limited preprocessing:

- Tokenization (converting words or word pieces into token ID)
- Sometimes lowercasing
- No removing of stop words

Gathering Training Data - Language Models

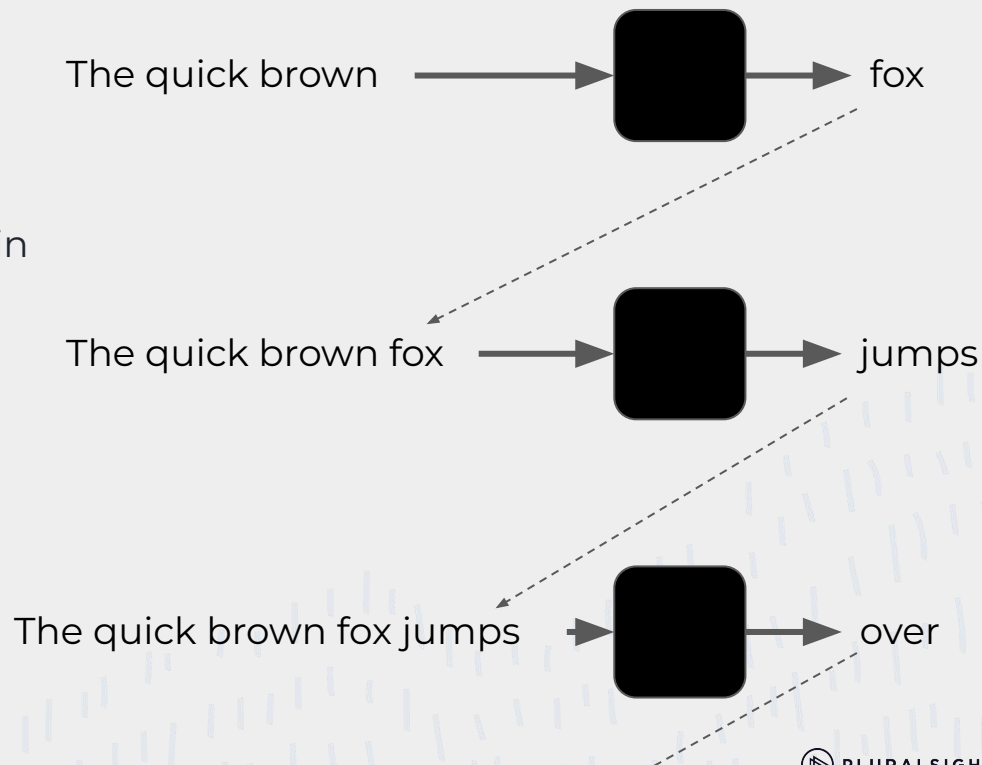
Some factors to consider when gathering training data:

- Tone, writing style
- Subject matter, domain knowledge
- Language representation

Choosing a Model: Language Models

Language models need to operate on **sequences** of **discrete** values, specifically in an autoregressive way. This is typically the realm of:

- Recurrent Neural Networks (RNN)
- Transformers



Choosing a Model: Language Models

RNN

RNNs take elements of an **arbitrary** length sequence as input **one at a time**, update a **hidden state** which acts as memory, and use the final hidden state to predict the next element.

Transformer

Transformers take a **fixed length** sequence as an input, pass information between all elements via **self attention**, and use the last vector to predict the next element.

Choosing a Model: Language Models

RNN

- Inputs are processed sequentially (slower)
- Context is limited to whatever fits in the hidden state at a given step
- Works on inputs of any length (in theory)
- Memory uses is constant

Transformer

- Inputs are processed in parallel (faster)
- Entire input is available as context
- Only inputs that fit in the context window can be utilized
- Memory use is quadratic in the context length

Transformers are currently the architecture of choice.

Choosing a Model: Language Models - Inputs

Regardless of the underlying model, text cannot be used directly by the model. It must first be **tokenized**. This maps elements of the input (characters, words, word pieces, emojis) to IDs in a **vocabulary**. Each **token** is then represented by a D-dimensional vector called an **embedding**.

Tokenization

The quick brown fox → [42, 1337, 73, 109]

Vocabulary Lookup

42 → [0.21, 0.97, ..., 0.59]

Choosing a Model: Language Models - Inputs

Embeddings are typically stored as a (vocab_size x D) matrix where the token ID is the corresponding row in the matrix.

These embeddings are initialized randomly and trained along with the model in an end-to-end fashion.

Tokenization

The quick brown fox → [42, 1337, 73, 109]

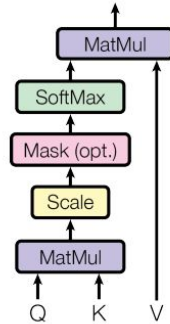
Vocabulary Lookup

42 → [0.21, 0.97, ..., 0.59]

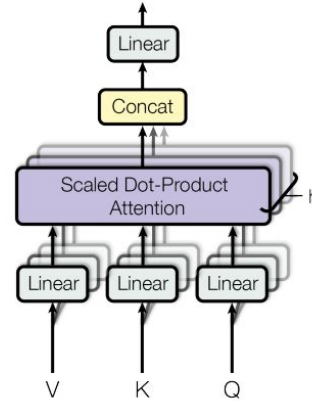
The Power of Attention Mechanism

Attention is a mechanism allowing the model to focus on relevant parts of the input sequence for generating output.

Scaled Dot-Product Attention



Multi-Head Attention

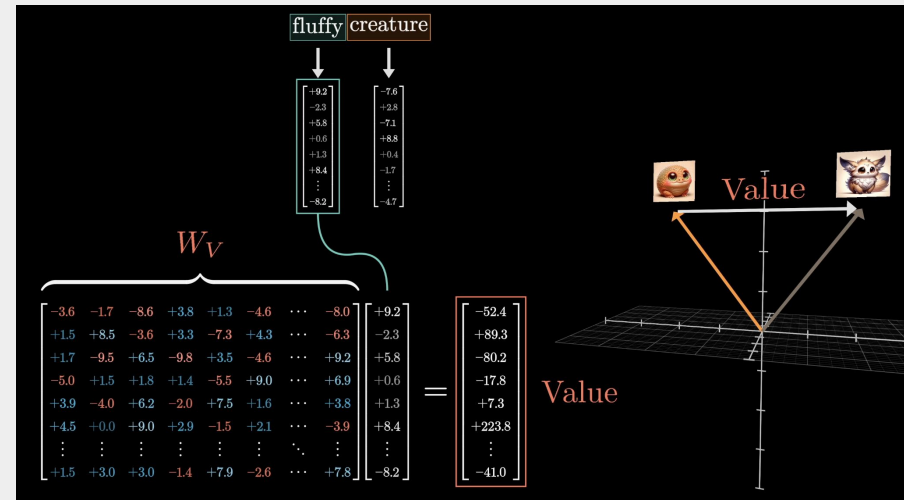


Choosing a Model: Language Models - Hidden

In the middle of the Transformer is a stack of **blocks** that 1) use **attention** to pass information between tokens, and 2) pass the updated tokens through a **MLP** to further process them.

For a very beautiful and in-depth explanation of the details, of this and many other math and neural network concepts, I highly recommend checking out

[3Blue1Brown's explanation.](#)

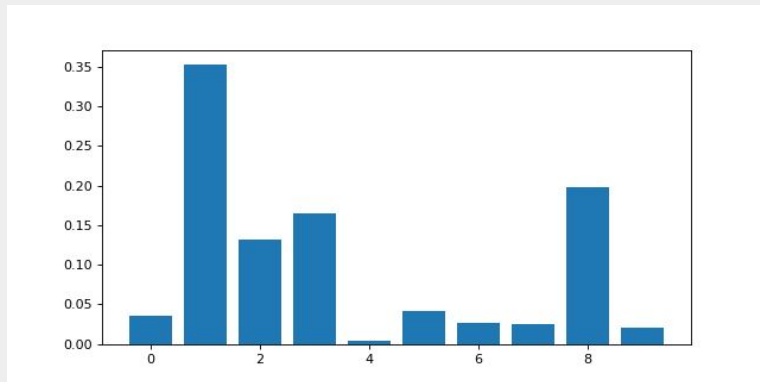


[Image credit](#)

Choosing a Model: Language Models - Outputs

The model output is a **probability distribution over its vocabulary**. This means the model produces a vocab_size vector with each element representing the probability of the token at a given index given the vocabulary (prob of token 0 ... token N).

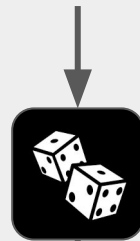
$$p(\text{vocab} \mid x)$$



Choosing a Model: Language Models - Outputs

In order to feed the output back into the model, this probability distribution must be **sampled** somehow (taking the argmax, weighted random choice, etc.).

$$p(\text{vocab} \mid x)$$



y

WORKING WITH MODERN LANGUAGE MODELS

Working with Language Models

Language models continue text in an autoregressive fashion. For something like ChatGPT, this is augmented by

- Fine tuning on the task of accepting and outputting conversational text
- Backend infrastructure for maintaining state across multiple interactions

GPT-3 vs. ChatGPT (and GPT-4)

GPT-3

- “Continue the thought”
- Zero-shot / few-shot tasks
- Not always intuitive to prompt
- Requires longer prompts to achieve conversational tasks
- Bad at math
- Hallucination

ChatGPT

- “Continue the thought”
- Zero-shot / few-shot tasks
- Explicitly trained to be conversational
- Requires shorter prompts to achieve conversational tasks
- Bad at math
- Hallucination

Working with Language Models: Zero-Shot

A **zero-shot** task is when you simply ask the model to do something without providing examples or additional domain knowledge. This relies purely on:

- The directions your prompt provides
- The built-in knowledge the model has from training

Working with Language Models: Zero-Shot

One approach to influencing generation is to use a template (a.k.a. madlib).

For example:

“As {a_persona} I want you to {do_something} so that {some_goal}. {acceptance_criteria}”

- “As a person giving a TED talk I want you to explain the importance of drinking enough water so that the reader will be encouraged to drink more. Be as concise as possible.”

Working with Language Models: Few-Shot

A few-shot task is when the prompt contains additional information and/or examples of the desired behavior. This helps to extend the model's capabilities to:

- Encourage the model to behave in a more specific manner
- Tasks that are best described by example
- Reasoning with data that may not be directly contained in the model's weights

Working with Language Models: Few-Shot

One task this approach allows is open-domain question answering. By providing enough information to directly in the prompt, the model can use its natural language reasoning capabilities to synthesize an answer, even if the subject matter was never part of the training data.

For example:

"Using the following information {background_information}, answer the following question: {question}"

"Using the following information 'Alice, Bob, and Cathy all live on the same street. Alice lives next to Bob, and Cathy is Bob's neighbor', answer the following question: are Alice and Cathy neighbors?"

Working with Language Models: Multi-Step

Sometimes models have enough background knowledge to solve a problem, but their default response structure can lead it down a path that does not reference it. Some approaches to solve this are [Chain of Thought](#), [Tree of Thought](#), and [Step-Back Reasoning](#).

For example (step-back prompting):

User: "Given the question '{question}', what background information would be useful to be able to answer this?"

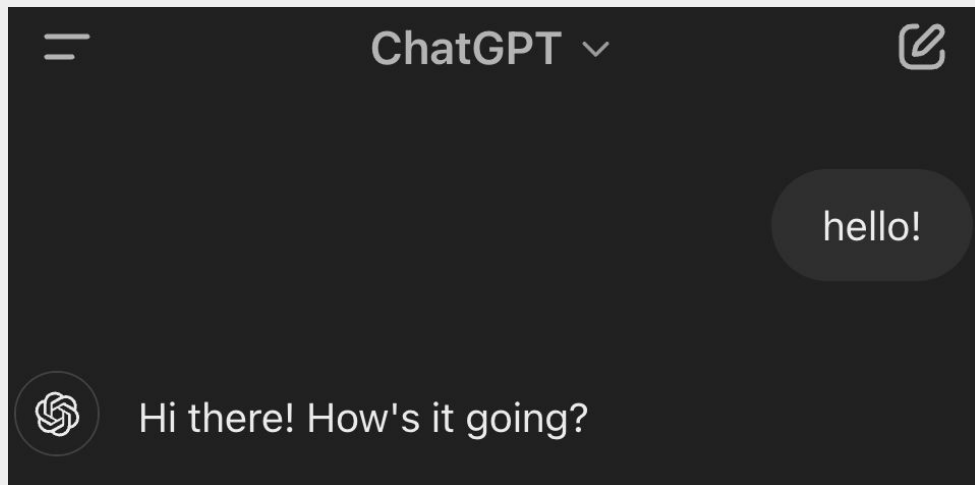
Agent: "<background information>"

User: "Given that, answer the question: '{question}'."

Agent: "<hopefully correct answer>"

Anatomy of a Chat

The ChatGPT web interface is really a frontend on top of the [API](#). This is a standard REST API that takes JSON as input and produces JSON as output. Let's look at how what we see in the app maps to the API.



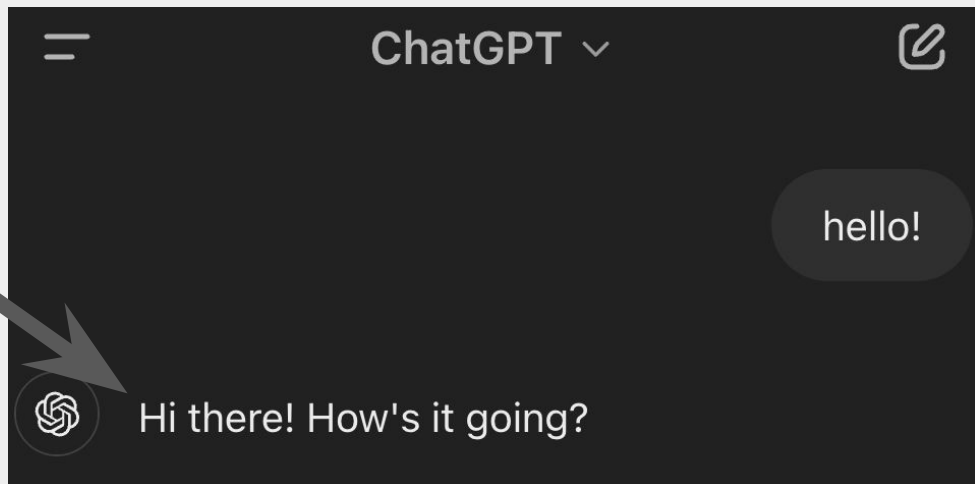
Anatomy of a Chat: Input

```
curl https://api.openai.com/v1/chat/completions \  
-H "Content-Type: application/json" \  
-H "Authorization: Bearer $OPENAI_API_KEY" \  
-d '{  
  "model": "gpt-4o",  
  "messages": [  
    {  
      "role": "system",  
      "content": "You are a helpful assistant." (hidden)  
    },  
    {  
      "role": "user",  
      "content": "hello!"  
    }  
  ]  
'
```



Anatomy of a Chat: Output

```
{  
  ...  
  "choices": [{  
    "index": 0,  
    "message": {  
      "role": "assistant",  
      "content": "\n\nHi there! How's it going?",  
    },  
    "finish_reason": "stop"  
  }],  
  "usage": {  
    "prompt_tokens": 9,  
    "completion_tokens": 12,  
    "total_tokens": 21  
  }  
}
```



Anatomy of a Chat: Roles

Chat based models define multiple **roles** for specifying particular behavior:

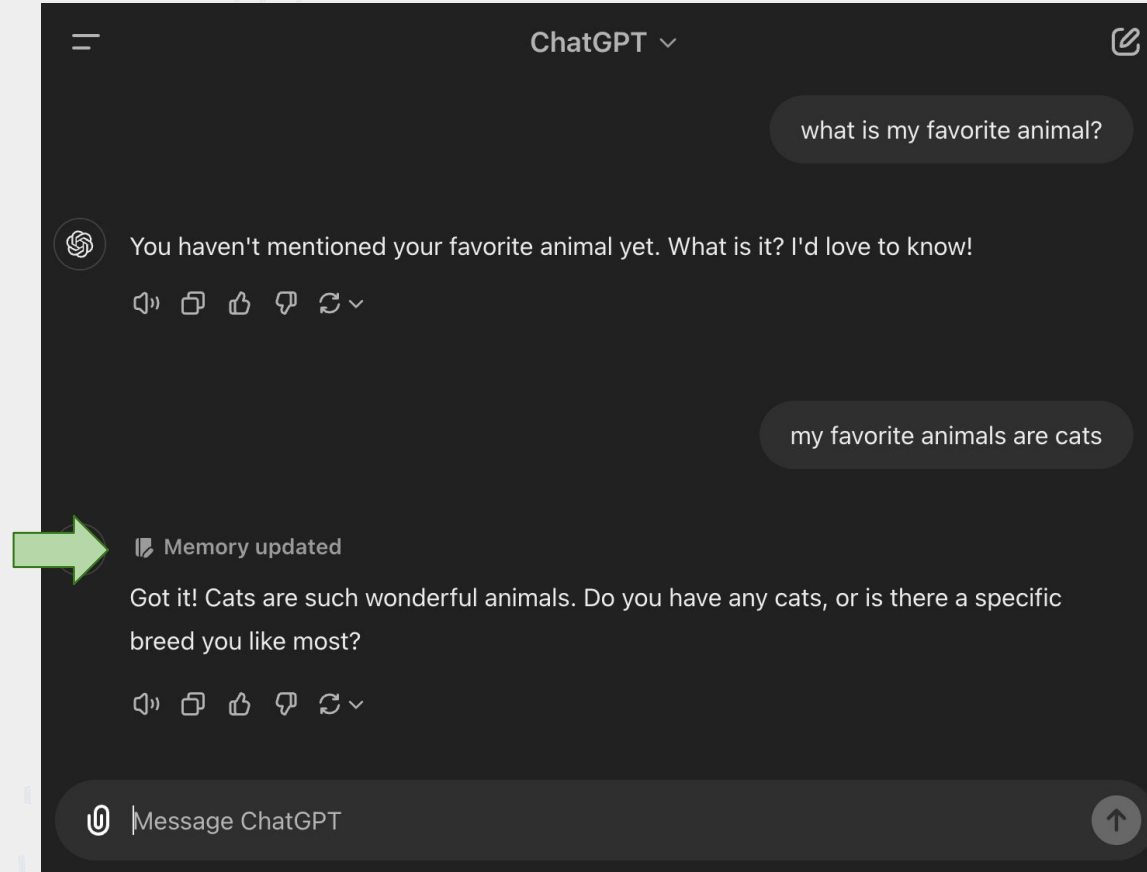
- **system:** special instructions provided to the model that the user cannot see nor change
- **user:** a message from the user to the model
- **assistant:** the response to the previous **user** message

Exercise: Implementing ChatGPT's Memory System

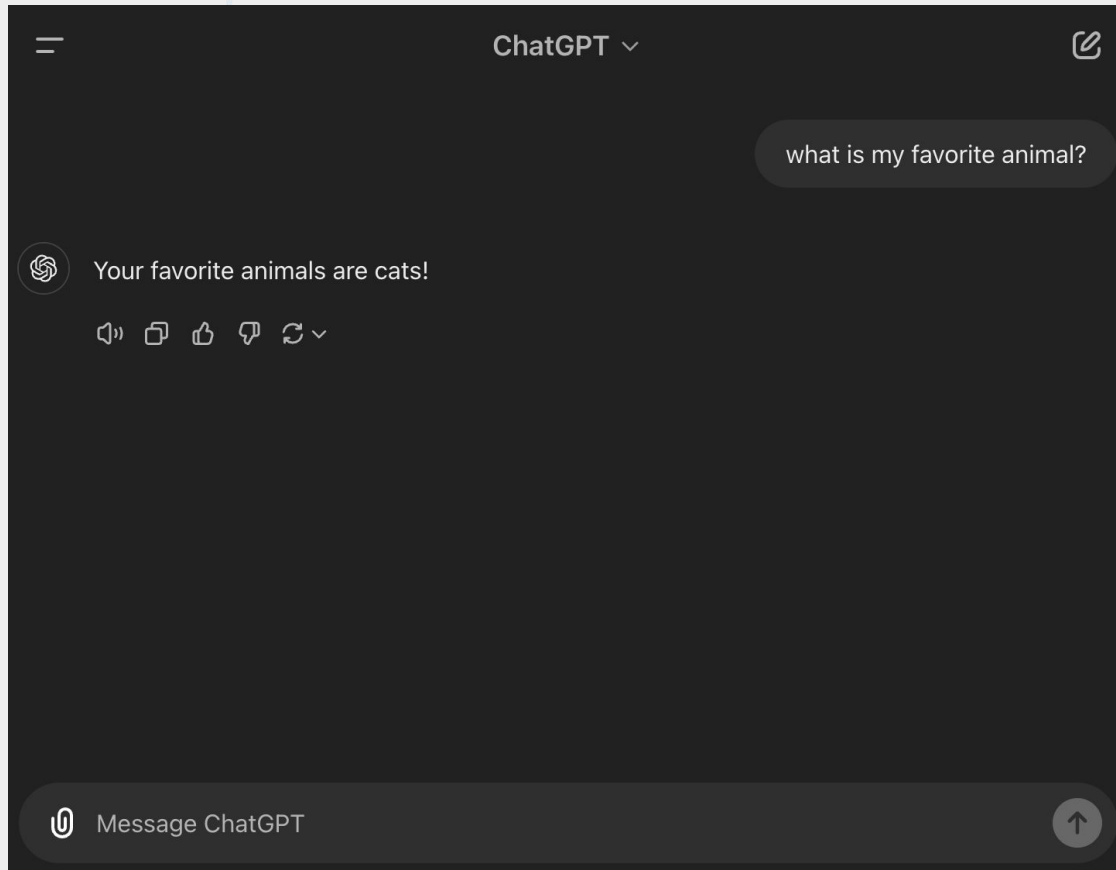
Let's implement a version of the memory system from the ChatGPT application. In doing so, we will gain experience with:

- Managing chat states
- Using prompting to implement a memory detector
- Identifying memories to the conversation

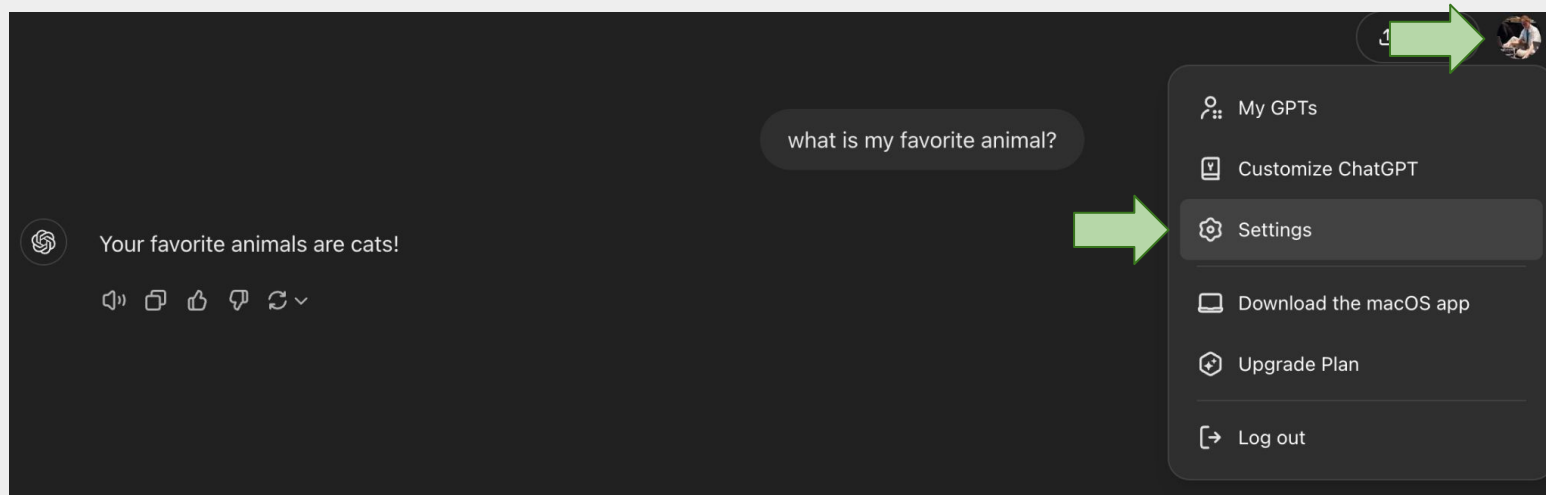
Exercise: Implementing ChatGPT's Memory System



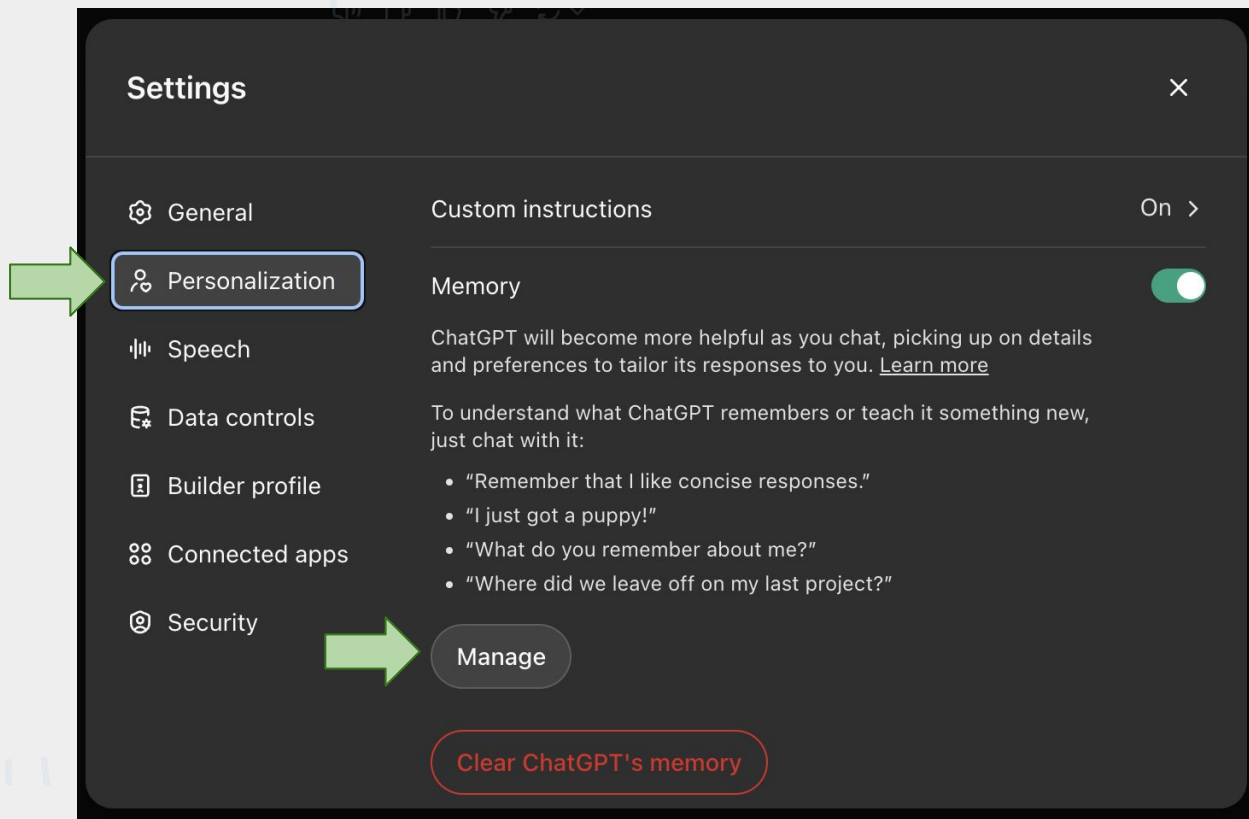
Exercise: Implementing ChatGPT's Memory System



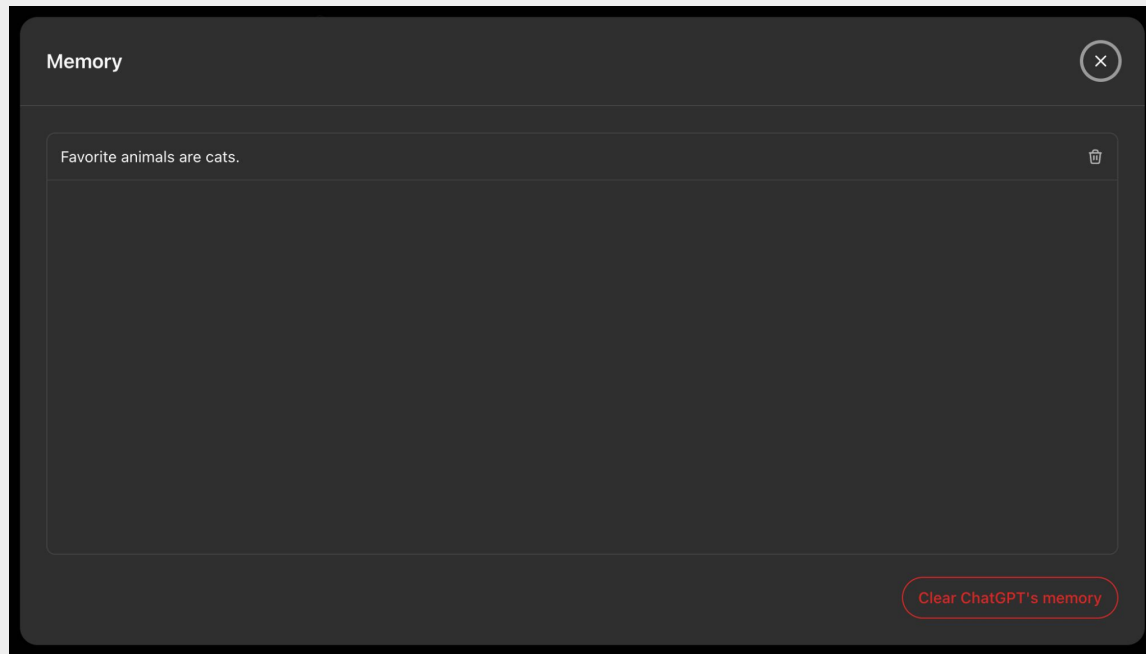
Exercise: Implementing ChatGPT's Memory System



Exercise: Implementing ChatGPT's Memory System



Exercise: Implementing ChatGPT's Memory System



Working with Language Models: Extensions

An extension to the few-shot approach is **retrieval-augmented generation**. This is actually the combination of a few things:

1. A model for creating embeddings from text with a notion of similarity
2. A k-nearest neighbour index for finding similar entries to a query
3. A database associating the embedding with metadata (e.g. the original text)
4. A language model

Working with Language Models: Extensions

The workflow is then:

1. Convert the question into an embedding
2. Query the k-nn, get the k most similar entries, and lookup their text
3. Add the retrieved text to the prompt along with the question for the model to answer

Working with Language Models: Extensions

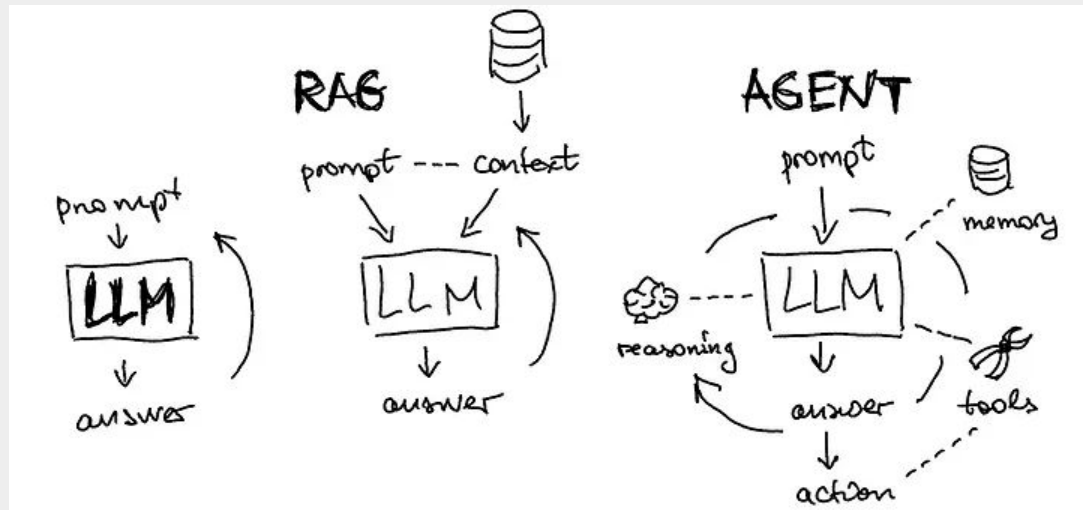
An other extension is **function calling** or **tool use**. With this approach:

1. The model is made aware of one or more functions/tools it can use, what they do, and what inputs they require.
2. The user makes a request that possibly requires calling a function.
3. The model outputs how to call the appropriate function with the provided inputs.

Example: [Gorilla](#), [DIY](#)

LLM vs. RAG vs. Agent

By now, we know that LLMs on their own work based on general knowledge. RAG systems extend them with data unseen at train time. **Agents** are what we call systems that can perform tasks on their own (sometimes with limited human input).



[Image Credit](#)

Agents

Generally Agents are defined by:

- Having memory and often RAG
- Tool use
- Planning and reasoning
- The ability to accomplish a task, rather than purely assisting

Agents

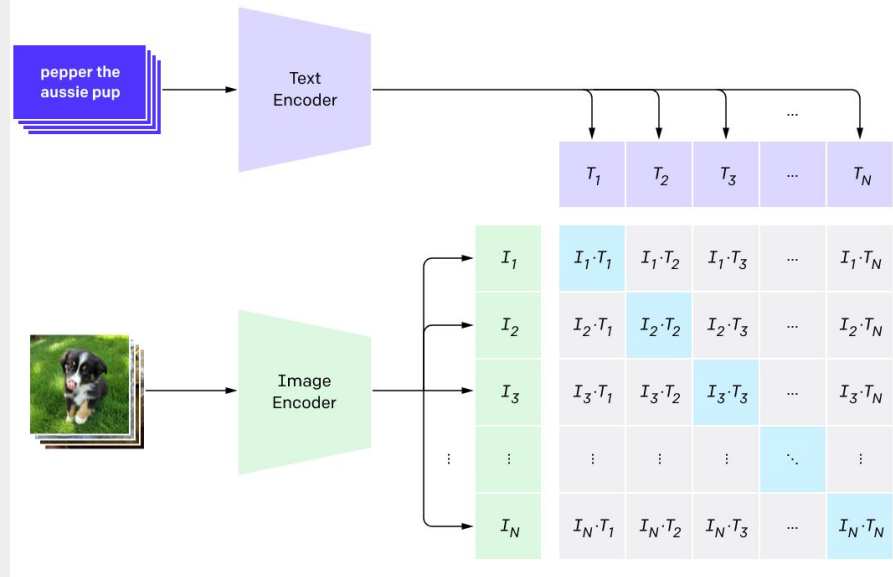
Some examples of Agents are:

- [Devin](#)
- [Pythagora](#)
- [AI Scientist](#)

Multimodality: Text / Image Similarity

CLIP from OpenAI, which calculates the similarity between text and images was used to train DALL-E. After being open sourced, it also led the wider community to create numerous text to image experiments and tools.

1. Contrastive pre-training



LLMs: Cost - OpenAI

Model	Pricing	Pricing with Batch API*
gpt-4o	\$5.00 / 1M input tokens	\$2.50 / 1M input tokens
	\$15.00 / 1M output tokens	\$7.50 / 1M output tokens
gpt-4o-2024-05-13	\$5.00 / 1M input tokens	\$2.50 / 1M input tokens
	\$15.00 / 1M output tokens	\$7.50 / 1M output tokens
gpt-3.5-turbo-0125	\$0.50 / 1M input tokens	\$0.25 / 1M input tokens
	\$1.50 / 1M output tokens	\$0.75 / 1M output tokens
gpt-3.5-turbo-instruct	\$1.50 / 1M input tokens	\$0.75 / 1M input tokens
	\$2.00 / 1M output tokens	\$1.00 / 1M output tokens

[link](#)

LLMs: Cost - Anthropic

Claude 3.5 Sonnet

- ✓ Our most intelligent model to date
- ✓ 200K context window

Input
\$3 / MTok

Output
\$15 / MTok

Claude 3 Opus

- ✓ Powerful model for complex tasks
- ✓ 200K context window

Input
\$15 / MTok

Output
\$75 / MTok

Claude 3 Haiku

- ✓ Fastest, most cost-effective model
- ✓ 200K context window

Input
\$0.25 / MTok

Output
\$1.25 / MTok

[link](#)

LLMs: Cost - Google

Free of charge*	Pay-as-you-go (prices in USD)***
Rate Limits** 15 RPM (requests per minute) 1 million TPM (tokens per minute) 1,500 RPD (requests per day)	Rate Limits** 1000 RPM (requests per minute) 4 million TPM (tokens per minute)
Price (input) Free of charge	Price (input) \$0.35 / 1 million tokens (for prompts up to 128K tokens) \$0.70 / 1 million tokens (for prompts longer than 128K)
Context caching Not applicable	Context caching \$0.0875 / 1 million tokens (for prompts up to 128K tokens) \$0.175 / 1 million tokens (for prompts longer than 128K) \$1.00 / 1 million tokens per hour (storage) Learn more
Price (output) Free of charge	Price (output) \$1.05 / 1 million tokens (for prompts up to 128K tokens) \$2.10 / 1 million tokens (for prompts longer than 128K)
Prompts/responses used to improve our products Yes Learn more Try it now in Google AI Studio	Prompts/responses used to improve our products No Learn more Set up billing in Google AI Studio

[link](#)

LLMs: Cost - Mistral

Open source models

		Input	Output
open-mistral-7b	A 7B transformer model, fast-deployed and easily customisable.	\$0.25 /1M tokens	\$0.25 /1M tokens
open-mixtral-8x7b	A 7B sparse Mixture-of-Experts (SMoE) . Uses 12.9B active parameters out of 45B total.	\$0.7 /1M tokens	\$0.7 /1M tokens
open-mixtral-8x22b	Mixtral 8x22B is currently the most performant open model. A 22B sparse Mixture-of-Experts (SMoE) . Uses only 39B active parameters out of 141B.	\$2 /1M tokens	\$6 /1M tokens

Optimized models

		Input	Output
mistral-small-2402	Cost-efficient reasoning for low-latency workloads.	\$1 /1M tokens	\$3 /1M tokens
codestral-2405	State-of-the-art Mistral model trained specifically for code tasks.	\$1 /1M tokens	\$3 /1M tokens
mistral-medium-2312	*Will soon be deprecated*	\$2.7 /1M tokens	\$8.1 /1M tokens
mistral-large-2402	Top-tier reasoning for high-complexity tasks. The most powerful model of the Mistral AI family.	\$4 /1M tokens	\$12 /1M tokens

LLMs: Cost

You can of course host your own models:

- [Amazon Bedrock](#)
- [Azure](#)
- [Google Vertex AI](#)

However this generally increases complexity and is often more expensive (depending on your exact usage). In cases where you need more control this is still a viable option.

Where to Find Models/Tools

Fortunately, there are many great models that either work out of the box or can be easily adapted to many common tasks:

- [scikit-learn](#)
- [NLTK](#)
- [spaCy](#)
- [Huggingface](#)

Discussion

Let's talk about various use cases, both front- and backend, that could take advantage of LLMs. What are the pros and cons of each?

ADAPTING PRE-TRAINED MODELS

Fine Tuning

Fine tuning is the process of taking a pre-trained (general purpose) model and training it further on data specific to a particular task.



Thank you!

If you have any additional questions, please ask!

